Eötvös Loránd Tudományegyetem

Informatikai Kar

Algoritmusok és Alkalmazásaik Tanszék

Cache-optimális algoritmusok elemzése

Szabó László Ferenc
Docens

Nagy Péter
Programtervező Informatikus MSc

Budapest, 2023

Tartalomjegyzék

1. Bevezetés **3**

2. Cache modellje **4**

2.1. Csere stratégiák 5

2.1.1. Online algoritmusok 6

2.1.2. Verem stratégiák 6

2.1.3. OPT 7

2.1.4. LRU 8

2.1.5. FIFO 8

2.2. Asszociativitás 9

2.3. Írási stratégia 10

3. Gyorsítótár független algoritmusok **12**

3.1. Lista 12

3.1.1. Másolás 12

3.1.2. Partíció 13

3.1.3. Leszámláló rendezés 14

3.2. Mátrix szorzás 15

3.2.1. Gyorsítótár tudatos 16

3.2.2. Kettő-hatvány négyzetes mátrixok 17

3.2.3. Gyorsítótár független 18

3.2.4. Strassen algoritmus 19

3.2.5. Z-sorrend 20

3.3. Rendezés 21

3.3.1. Összefésüléses rendezés 21

3.3.2. Tölcsér rendezés 23

3.4. Edénysor szita 25

3.4.1. A rekurzív fa 26

3.4.2. Rekurzív szita 27

3.4.3. Műveletigény gyorsítótár nélkül 27

3.4.4. A műveletigény alsó korlátja 31
3.4.5. Gyorsítótár független edény szita 33
3.4.6. Gyorsítótár tudatos edény szita 36
3.4.7. Műveletigény gyorsítótárral 36
3.4.8. Edénysor szita 40

4. Szimulátor **44**
4.1. C programok 44
4.1.1. Mátrix szorzás 46
4.1.2. Rendezés 47
4.1.3. RekurzívSzita 47
4.2. RISC-V szimulátor 48
4.3. Gyorsítótár szimulátor 51
4.4. Összesítő 54
4.5. Fordítás 54
4.6. Tesztek 55
4.7. Dokumentáció ??? 55

5. Mérési eredmények **56**
5.1. Mátrix szorzás 56
5.2. Rendezés 57
5.3. RekurzívSzita 57

6. Összegzés **59**

7. Irodalomjegyzék **60**

1. fejezet

Bevezetés

A modern számítógépek olyan sebességeket érnek el, hogy a jelterjedés fizikai korlátai már egy integrált áramkörön belül sem feltétlenül engedik meg az összes áramköri elem szinkron működést. A végrehajtó egység és a memória sebessége között már nagyságrendi különbségek vannak.

A probléma kezelésére gyorsítótárakat vezettek be a végrehajtó egység és a memória közé, ami az utoljára használt memóriaelemeket tárolja. A gyorsítótár sebessége nagyobb mint a memóriáé, de lassabb, mint a végrehajtó egységé. Egy modern számítógépben a gyorsítótáraknak hierarchiája is van, ahol egy gyorsítótár egy szintben alatta lévő másik gyorsítótár elemeit tárolja.

A gyorsítótárakkal ellátott számítógép legjobb kihasználásához olyan algoritmusokat kell tervezni, amik figyelembe veszik a gyorsítótárak hierarchiáját.

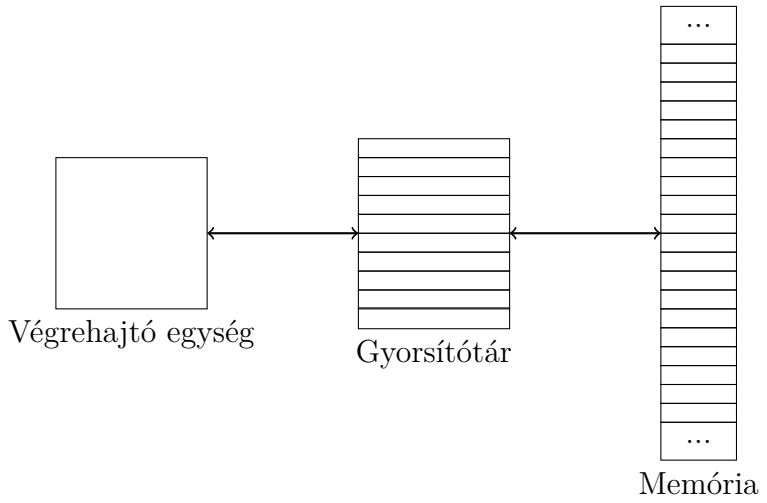
A szakdolgozatban bemutatom az eratoszthenészi szita egy gyorsítótár független, és egy gyorsítótár tudatos változatát, melyek hatákony megoldást adnak egy intervallumba eső számok prímfaktorizációjára, akár többszintű memóriahierarchiában is.

2. fejezet

Cache modellje

Az ideális gyorsítótár modell[1] egy végrehajtó egységből, memóriából, és egy gyorsítótárból áll. A végrehajtó egység a memóriát csak a gyorsítótáron keresztül éri el.

A végrehajtó egység a memóriát gépi szavanként éri el, ez egy kicsi konstans, például 8 bájt. Egy (Z, L) gyorsítótár L hosszú gyorsítótársorokból áll, és Z/L sort tartalmaz. Egy sor L gépi szót tárol, ezek a memóriában folytonosan helyezkednek el. Egy sor címe a memóriába csak a sorhossz többszöröse lehet. A gyorsítótár a memóriát gyorsítótársoronként tárolja. Egy sor a gyorsítótárban a memória bármelyik szakaszát tárolhatja.



2.1. ábra. Az ideális gyorsítótár modell

Amikor a végrehajtó egység el akar érni egy adott szót a memóriában, a gyorsítótárhoz fordul. Ha a tárban van olyan sor, ami tartalmazza a kért gépi szót, akkor a gyorsítótár a kérést azonnal kiszolgálja. Ezt gyorsítótár-találatnak nevezzük. Ha a tár egyik sora se tartalmazza a kért szót, akkor a szóhoz tartozó sort a tár be kell olvassa a memóriából, majd ezután tudja teljesíteni a kérést. Ezt gyorsítótár-hibának

nevezzük.

Ha a tárban nincs üres sor amikor egy új sort be kell olvasni, akkor először egy sort ki kell ürítsen. Ha az ürítésre kiválasztott sor módosult a hozzá tartozó memóriához képest, akkor azt először vissza kell írnia a memóriába. Miután a sort szinkronizálta a memóriával, az új sorral felülírja az ürített sort.

A gyorsítótár-csere stratégiája diktálja, hogy a tár hogyan választja ki a sort, amit ürít. Az ideális gyorsítótár modell ürítési stratégiája az optimális offline stratégia[2]. Az optimális stratégia azt a sort üríti, amit a jövőben legkésőbb ér el a program a teljes futása során. Ez a stratégia használja ki a legjobban az időbeli lokalitást.

A gyorsítótár modellben az időbonyolultság az algoritmus futása alatt végzett elemi műveletek száma, a memóriaelérésekkel együtt. A gyorsítótár-bonyolultság a futás alatti találati hibák száma.

Egy algoritmus gyorsítótár-tudatos, ha van olyan fordítási vagy futási idejű paramétere, ami a gyorsítótár méretétől függ. Ha nincs ilyen paramétere, akkor az algoritmus gyorsítótár-független.

2.1. Csere stratégiák

Több csere stratégia is ismert:

- MIN, minimális, OPT, optimális, vagy Bélády algoritmusa: Ezt az elemet cseréli, amelyiket a jövőben legkésőbb éri el az algoritmus. Ennek eldöntésére szükség van a teljes memóriaelérési sorozatra.

- LRU, least recently used, legrégebben használt: Azt a sort cseréli, amit a legrégebben ért el az algoritmus. Ennek eldöntésére elég a csere pillanatáig ismerni az elérési sorozatot. Az LRU heurisztikája az a feltételezés, hogy az időben közelebbi elért memóriát valószínűleg újra el kell érni.

- LFU, least frequently used, legritkábban használt: A gyorsítótár nyilván tartja, hogy egy sort hányszor ért el az algoritmus mióta a gyorsítótárba került. Azt az elemet cseréli, amelyik elérési száma a legkisebb.

- FIFO, first in, first out, első be, első ki: Azt a sort cseréli, amelyik legrégebben került a tárba. A stratégia előnye az egyszerű megvalósíthatóság.

- Random, véletlen: A cserét véletlenszerűen választja ki. Az előnye az egyszerű implementáció.

Ha S egy gyorsítótár-csere stratégia, akkor legyen S(k) egy olyan gyorsítótár, ami az S stratégiát használja, és k sort tartalmaz.

2.1.1. Online algoritmusok

A gyakorlatban megvalósítható gyorsítótár-csere stratégiák online algoritmusok. Az online algoritmusok a bemenetüket sorban, egyesével kell feldolgozzák, a teljes bementet nem látják egyszerre. Egy gyorsítótár-csere algoritmusnak minden memória-eléréskor úgy kell döntést hozzon a cseréről, hogy csak a múltbeli elérések sorozatát ismeri.

Egy offline algoritmus a teljes bemenethez hozzáfér a teljes futása során.

A stratégiák elemzését egy egyszerűbb modellben[5] végezzük. Ezek a szabályok az igény szerinti lapozást írják le. Legyen $A = [1, 2, ..., n]$ az elérhető elemek halmaza. Legyen X egy A feletti véges sorozat, az elérések sorozata. Legyen $B_t(k)$ az $S(k)$ gyorsítótár tartalma X t hosszú prefixének feldolgozása után. Minden k-ra:

- Legyen $B_0(k) = \emptyset$. Tehát a gyorsítótár üres az algoritmus futása előtt.
- Ha $t > 0$, és $X_t \in B_{t-1}(k)$, akkor legyen $B_t k = B_{t-1}(k)$. Azaz találat esetén a gyorsítótár nem változtatja meg, hogy mely elemeket tartalmazza.
- Ha $t > 0$, $X_t \notin B_{t-1}(k)$, és $|B_{t-1}(k)| < k$, akkor legyen $B_t k = B_{t-1}(k) \cup \{X_t\}$. Ha van szabad hely a gyorsítótárban, akkor találati hiba esetén csere nélkül hozza be a memóriaelemet a tárba.
- Ha $t > 0$, $X_t \notin B_{t-1}(k)$, és $|B_{t-1}(k)| = k$, akkor legyen $y_t \in B_{t-1}(k)$, és $B_t(k) = B_{t-1}(k) \setminus y_t \cup \{X_t\}$. Találati hiba esetén ha nincs üres sor a tárban, akkor egy elemet kicserél, hogy behozhassa az elérni kívántat. A cserélendő elem az y_t, ezt a csere stratégia választja ki.

Ebben a modellben a gyorsítótárban nem sorok vannak, hanem A elemei. Ezek a gyorsítótár modellben a memória azon tartományai, amik gyorsítótár sorok lehetnek. Valamint az elérésben nem különböztetjük meg az olvasást és az írást.

2.1.2. Verem stratégiák

Egy S gyorsítótár-csere stratégiát verem algoritmusnak[5] nevezünk, ha teljesíti a következő feltételt. Legyen X egy memória-elérési sorozat. S verem algoritmus, ha minden $t \in 1, 2, ..., |X|$, és minden $k \in \mathbb{N}$-re

$$B_t(k) \subset B_t(k+1)$$

Azaz ha minden időpontban egy nagyobb gyorsítótár tartalmazza az összes kisebb gyorsítótár elemeit.

A verem algoritmusnak egy ekvivalens definíciója, ha S X futtatása közben minden $t \in 1, 2, ..., |X|$ időpontra ad egy egyértelmű preferencialistát az $\{X_1, X_2, ..., X_t\}$-ben előforduló elemekre, a gyorsítótár méretétől függetlenül, és egy $S(k)$ gyorsítótár csere esetén $B_t(k)$ elemei közül a legkisebb precedenciájút cseréli. A precedencialista nem feltétlen kell megegyezzen minden időpontban.

Egy verem algoritmust használva a gyorsítótár méretét növelve a találtai hibák száma nem nő. Nézzünk t időpontban egy gyorsítótár találatot $S(k)$-ban. Mivel $B_t(k) \subset B_t(k+1)$, ezért az elérés $S(k+1)$-ben is találat. Tehát minden k-nál nagyobb gyorsítótárban is találat az elérés.

Az LRU verem algoritmus.

Az OPT verem algoritmus.

Az LFU verem algoritmus, feltéve, hogy az egyenlő frekvenciájú elemekre minden időpontban létezik egy egyértelmű precedencia.

A FIFO nem verem algoritmus.

A véletlen startégi nem feltétlen verem algoritmus. Ha egyenletesen véletlenül választjuk ki egy $S(k)$ gyorsítótár elemét cserére, akkor pozitív a valószínűsége annak, hogy az $S(k+1)$ tár egy másik elemet választ. Ha a t időpontban a tár méretétől függetlenül egyenletes véletlenül kiválasztjuk $X_1, X_2, ..., X_t$ egyedi elemeinek egy permutációját, és ezt precedencialistaként használjuk a cserélt elem kiválasztására, akkor a véletlen stratégia verem algoritmus. Ez az RR stratégia. Egy adott $RR(k)$ gyorsítótárban egy elem ürítésének esélye $1/k$, tehát egyenletes eloszlású.

2.1.3. OPT

Az optimális stratégiánál jobb eredményt lehet elérni, ha nem követeljük meg, hogy egy elérést csak a gyorsítótárból lehet kiszolgálni. Nézzük a $X = (1, 2, 3, 1, 2)$ elérési sorozatot, a $t = 3$ időpontot, és egy kételemű gyorsítótárat. A gyorsítótár ekkor tele van, és $B_2(2) = \{1, 2\}$.

Nézzük az $OPT(2)$ stratégiát. A 3 tárba hozatalához egy elemet ki kell cserélnie. Cserére a 2-t választja, mert ez az elem lesz legkésőbb újra elérve. Így $B_3(2) = \{1, 3\}$. $t = 4$-ben az elérés találat, de $t = 5$-ben 2 már nincs a tárban, és találati hibát okoz. Tehát X kiszolgálásához 4 memóriaműveletre volt szükség, a $t \in \{1, 2, 3, 5\}$ időpontokban.

Nézzünk egy olyan optimális stratégiát, ahol csere nélkül, közvetlenül a memóriából is ki lehet szolgálni egy elérést. Ez a stratégia $t = 3$-ban a 3 elérést a memóriából közvetlenül szolgálja ki, és $B_3(2) = B_2(2) = \{1, 2\}$. Ekkor X kiszolgálásához elég 3 memóriaművelet, a $t \in \{1, 2, 3\}$ időpontokban.

2.1.4. LRU

Egy adott problémára a versenyképességi analízisben egy A online algoritmust hasonlítunk össze az O optimális offline algoritmussal. Legyen X egy inputsorozat, $A(X)$ az A műveletigénye X-en, és $O(X)$ az O műveletigénye X-en. Ekkor A k-versenyképes[6], ha létezik egy C konstans, hogy minden X inputsorozatra

$$A(X) \leq kO(X) + C$$

Ha $C = 0$, akkor A szigorúan k-versenyképes. A legkisebb ilyen k az A versenyképességi aránya.

Slater és Tarjan[7] megmutatta, hogy az LRU(k) az OPT(k^*)-hoz képest $k/(k - k^* + 1)$ versenyképes. Ha $k = k^*$, akkor az LRU k-versenyképes az OPT-hoz képest. Ha $k = 2k^*$, akkor az LRU(k) az OPT(k*)-hoz képest

$$k/(k - k^* + 1) = 2k^*/(2k^* - k^* + 1) = 2k^*/(k^* + 1) < 2$$

versenyképes[1].

Legyen A egy adott algoritmus. Legyen $LRU(n, Z, L)$ a gyorsítótár komplexitása A-nak egy n méretű bemeneten és egy LRU(Z, L) gyorsítótáron. Legyen $OPT(n, Z, L)$ a gyorsítótár komplexitása A-nak egy n méretű bemeneten és egy OPT(Z, L) gyorsítótáron. Ekkor az $LRU(n, Z, L)$ gyorsítótár komplexitás reguláris[1], ha

$$LRU(n, Z, L) = O(LRU(n, 2Z, L))$$

Tegyük fel, hogy A egy reguláris gyorsítótár komplexitású algoritmus. Ekkor

$$OPT(n, Z, L) \leq LRU(n, Z, L) = O(LRU(n, 2Z, L)) = O(OPT(n, Z, L)$$

az optimalitást, a regularitást, és a 2-versenyképességet felhasználva. Tehát

$$LRU(n, L, Z) = \Theta(OPT(n, L, Z)$$

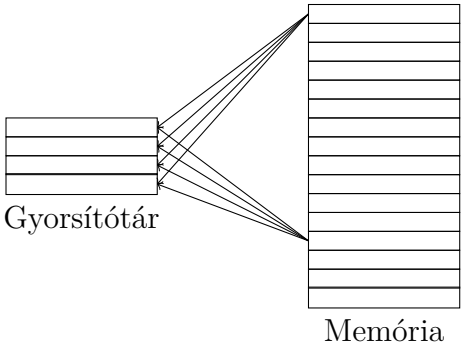
2.1.5. FIFO

Bélády[3] megmutatta, hogy FIFO stratégiára létezik olyan k és K, $k < K$, és egy elérési sorozat, hogy ezen az elérési sorozaton a FIFO(K) több találati hibát eredményez, mint a FIFO(k). Fornai és Iványi megmutatta[4], hogy a hiba arányának nincs felső korlátja.

2.2. Asszociativitás

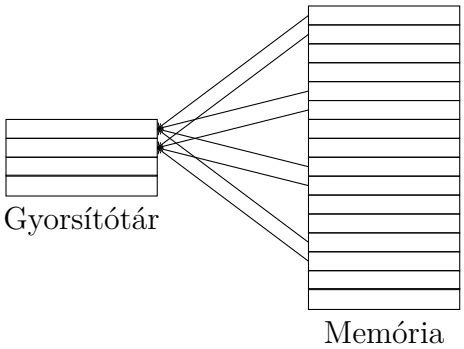
Az ideális gyorsítótár modellben a memória egy sorát a gyorsítótár bármelyik sorába be lehet hozni. Ezt hívják teljesen asszociatív gyorsítótárnak[8].

A gyakorlatban egy teljesen asszociatív gyorsítótár csak kevés sorra valósítható meg hatékonyan, mert minden elérésnél a gyorsítótárnak meg kell keresnie a sorok között, hogy tárolja-e az elérni kívánt sort.



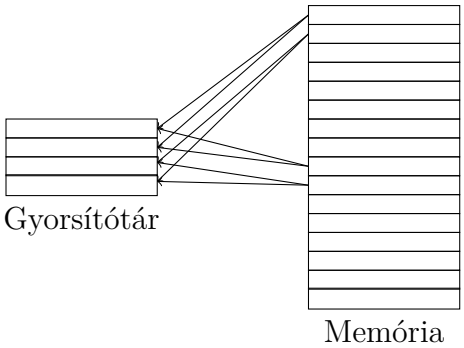
2.2. ábra. Teljesen asszociatív gyorsítótár

Közvetlen leképezésű gyorsítótárban[8] a memória egy sora a tárban csak egyetlen sorba hozható be. Ez a csere stratégiát is meghatározza, a beolvasni kívánt sor címe meghatározza a cserélt sort. Általában egy sor címét a gyorsítótárban a sor címének alsó bitjei határozzák meg, így a gyorsítótárba lehet hozni a memória egy egybefüggő szakaszát.



2.3. ábra. Közvetlen leképezésű gyorsítótár

Csoport-asszociatív gyorsítótárban[8] a memória egy sora a gyorsítótár több sorába is behozható, de nem az összesbe. A gyorsítótár több, egymástól független teljesen asszociatív gyorsítótárból áll. A memóriasor címe meghatározza, hogy a sor melyik asszociatív tárba kerül, és az asszociatív táron belül a csere stratégia alapján kerül elhelyezésre. Ha egy memóriasor A különböző gyorsítótár sorba kerülhet, akkor azt A-utas gyorsítótárnak nevezzük.



2.4. ábra. Csoport-asszociatív gyorsítótár

Tegyük fel, hogy a gyorsítótár paraméterei kettő-hatványok. A cache $Z = 2^z$ bájt, egy sor $L = 2^l$ bájt, és a cache $A = 2^a$ asszociatív, azaz egy memóriasor A gyorsítótár sorba kerülhet. Ekkor egy memóriabájt címének bitjeit egy lehetséges felosztása:

- Az l darab legkisebb helyi-értékű bit a bájt címe az őt befoglaló sorban.
- A következő $z-a$ helyi-értékű bit határozza meg, hogy a memóriasor a $C/(AL)$ darab teljesen asszociatív gyorsítótár közül melyikhez tartozik.
- A cím maradék bitjei meghatározzák, hogy egy teljesen asszociatív gyorsítótárhoz hány különböző sor tartozik.
- Ha $A = C/L$, akkor a gyorsítótár teljesen asszociatív.
- Ha $A = 1$, akkor a gyorsítótár közvetlen leképezésű.
- Ha $1 < A < C/L$, akkor a gyorsítótár A-utas.

2.3. Írási stratégia

A gyorsítótáraknak az írási műveleteket is kezelniük kell. Erre több különböző megoldás ismert[8].

Egy ha az írás olyan sort érint, amit a gyorsítótár tartalmaz, akkor az írási találat. Ilyenkor két megoldás közül lehet választani.

- A visszaírási (write back) stratégia az írást csak a gyorsítótárban végzi el, és az érintett sort megjelöli módosítottnak. Egy módosított sort a csere előtt a memóriába ki kell írni. Ezzel a megoldással memóriaműveletet lehet spórolni, ha ugyanarra a memóriasorra rövid időn belül több írás is történik. Ez a lehetséges hatékonyságnövelés növeli a megvalósítás áramköri komplexitását.

- A átmenő (write through) stratégia az írást a gyorsítótárban és a memóriában is elvégzi. A stratégia megvalósítása egyszerűbb, de minden írási művelet memóriaművelettel jár.

Ha az írási találati hibát is kétféleképpen kezelheti a gyorsítótár.

- Foglalással (write allocate) a gyorsítótár sorcserével a tárban is elhelyezi az írt sort. Ha az írás nem fedi le az egész memóriasort, akkor a gyorsítótár először az egész sort ki kell olvassa a memóriából, az olvasási találati hibához hasonló módon.

- Foglalás nélkül (no write allocate) a gyorsítótár az írást közvetlenül a memóriában végzi el, a saját tárát megkerülve. Ha a gyorsítótár ki kell tudjon szolgáljon olyan írásokat, amik nem a teljes memóriasort módosítják, akkor a memóriának támogatnia kell a memóriasornál finomabb elérési műveleteket.

Ezeknek a lehetőségeknek minden kombinációja megvalósítható, különböző bonyolultsággal.

- A módosított gyorsítótár sorok megnövelik a gyorsítótár állapotainak számát, és a műveletek állapotgépeinek állapotait is.

- A közvetlen memóriaírás a gyorsítótár megkerülésével szükségessé teszi, hogy a memóriát ténylegesen olyan finomsággal lehessen címezni, amilyen finoman a végrehajtó egység logikailag tudja címezni.

- Ha vannak párhuzamos gyorsítótárak, akkor ezek a megoldások a gyorsítótár koherenciára adott megoldást is befolyásolják. Párhuzamos gyorsítótárak előfordulhatnak egy végrehajtó egységgel is, például külön adat és utasítás gyorsítótár, és több végrehajtó egységgel rendelkező rendszerben is.

3. fejezet

Gyorsítótár független algoritmusok

3.1. Lista

Legyen A egy n elemű, N bájt hosszú tömb. A tömb elemei a memóriában folytonosan legyenek elrendezve. A feladat a tömb minden elemét sorban egyszer elérni. Ez az elérés lehet olvasás vagy írás is.

Tegyük fel, hogy a gyorsítótár mérete (Z, L), és létezik legalább egy gyorsítótársor, $Z \geq L$.

Ekkor a tömb minden elemeinek elérésének gyorsítótár bonyolultsága $O(1 + N/L)$. Az tömb eléréséhez legfeljebb 2 olyan memóriasort kell elérni, ahol a tömb nem fedi le az egész sort, a tömb elején és a tömb végén. A tömb többi eleme legfeljebb N/L memóriasort fed le teljesen.

3.1.1. Másolás

Legyen A egy n elemű, N bájt hosszú tömb. A tömb elemei a memóriában folytonosan legyenek elrendezve. A feladat a tömb minden elemét egy új memóriacímre másolni. A másolás történhet a tömbben előre és hátrafele is, attól függően, hogy a tömb régi és új helye hogyan fed át.

Tegyük fel, hogy a (Z, L) méretű gyorsítótárban legalább 2 gyorsítótársor van.

Ekkor az egyik gyorsítótársort az írásra használva, és a másik sort az olvasásra használva a másolás elvégezhető $2O(1 + N/Z) = O(1 + N/Z)$ memóriaművelettel.

3.1.2. Partíció

Legyen A egy n elemű, N bájt hosszú tömb. A tömb elemei a memóriában folytonosan legyenek elrendezve. Legyen T eldönthető tulajdonság, ami értelmezett a tömb minden elemére.

Rendezzük át helyben a tömböt úgy, hogy az összes olyan elem, amire a T tulajdonság hamis, a tömb elejére kerül, és azok az elemek, amikre a T tulajdonság igaz, a tömb végére kerülnek. A felosztás eredménye legyen a hamis tulajdonságú elemek száma.

A feladat a gyorsrendezés egy részfeladata, ahol a rendezendő tömb elemeit egy tengelyelem szerint két részre osztja, a tengelynél nem-nagyobbakra, és a tengelynél nagyobbakra. [9]

Algoritmus 3.1 Az $A[0 : n-1]$ tömb felosztása a T tulajdonság szerint

Require: A: a felosztandó tömb
Require: n: a tömb hossza
Require: T: egy eldönthető tulajdonság
Ensure: m: a hamis tulajdonságú elemek száma
Ensure: $A[0 : m-1]$ minden eleme hamis tulajdonságú T szerint
Ensure: $A[m : n-1]$ minden eleme igaz tulajdonságú T szerint
Ensure: az A kimenet az A bemenet permutációja

```
function FELOSZTÁS(A, n, T) → m
    l ← 0
    r ← 0
    while true do
        while (l + r < n) ∧ (¬T(A[l])) do
            l ← l + 1
        end while
        while (l + r < n) ∧ T(A[l]) do
            r ← r + 1
        end while
        if l + r ≥ n then
            return l
        end if
        t ← A[l]
        A[l] ← A[r]
        A[r] ← t
        l ← l + 1
        r ← r + 1
    end while
end function
```

Tegyük fel, hogy a gyorsítótár mérete (Z, L), és Z/L legalább akkor, hogy 3 tömbelemet a gyorsítótárban lehessen tartani, a memória bárhol elhelyezkedve. Mindig tároljuk a gyorsítótárban

- az l által vizsgált elemet,
- az r által vizsgált elemet,
- a cserénél használt ideiglenes t elemet.

Ekkor a felosztást legfeljebb $O(1 + N/L)$ memóriaművelettel el lehet végezni.

- Növekvő indexekkel legfeljebb a tömb összes memóriasorait egyszer járjuk be. Ez $O(1 + N/L)$ memóriaművelet.
- Csökkenő indexekkel legfeljebb a tömb összes memóriasorait egyszer járjuk be. Ez $O(1 + N/L)$ memóriaművelet.
- Az ideiglenes változóhoz tartozó memóriasorokat legfeljebb egyszer kell a gyorsítótárba hozni. Ez $O(1)$ memóriaművelet.

3.1.3. Leszámláló rendezés

A leszámláló rendezés egy lineáris műveletigényű, nem összehasonlításon alapuló rendezés.[9]

Legyen A egy n elemű, N bájt hosszú tömb. Legyen K egy kiszámítható függvény, ami a tömb minden elemét a $\{0, 1, ..., k - 1\}$ halmazba képzi. k legalább 2. Tipikusan k lényegesen kisebb, mint n. K az elemek kulcsát adja meg, ami szerint a tömböt rendezni szeretnénk.

A feladat a tömböt úgy átrendezni helyben, hogy sorban először az összes 0 kulcsú elem legyen, majd ezeket sorban kövesse az összes 1 kulcsú elem, és így tovább, végül a tömb végén a $k - 1$ kulcsú elemek helyezkedjenek el. A kimenetben adjuk meg minden lehetséges kulcsra azt is, hogy a tömbben hol helyezkednek el az adott kulcsú elemek.

A megoldás a tömb kétszeri bejárásával megadható.

- Inicializáljunk a $C[0 : k - 1]$ tömböt nullákkal, majd a tömb egyszeri bejárásával számoljuk meg a kulcsok frekvenciáját. Ez a C tömb a kimenet része is.
- Az $S[0 : k - 1]$ tömbben számítsuk ki a kulcsokhoz tartozó kezdőelemek indexeit. $S[0] = 0$, és $\forall i \in \{1, 2, ...k - 1\} : S[i] = S[i - 1] + C[i - 1]$. Ez az C és S egyszeri bejárásával kiszámítható. S a kimenet része.
- Másoljuk le S-t T-be. A következő dupla-ciklusban a T invariánsa a következő. Minden $k' \in \{0, 1, ..., k - 1\} : \forall i \in [S[k'], T[k']) \rightarrow K[A[i]] = k'$, azaz $S[k']$ és $T[k']$ között minden elem a helyén van.

- Járjuk be sorra k'-vel a $\{0, 1, ..., k-1\}$ kulcsokat. Minden k' kulcsra biztosítsuk, hogy a k'-höz tartozó szakasz minden eleme a megfelelő helyen van. Amíg $T[k'] - S[k']$ nem egyenlő $C[k']$-vel, addig legyen $i = T[k']$, és $k^* = K(A[i])$. Ha $k^* = k'$, akkor az $A[i]$ elem a helyén van, növeljük meg eggyel $T[k']$-t. Különben cseréljük meg $A[i]$-t és $A[T[k^*]]$-ot és növeljük meg eggyel $T[k^*]$-ot.

A megoldásban az utolsó ciklus minden lépésben egy elemet a helyére tesz, a ciklus invariánsa szerint, és egy elemet legfeljebb kétszer vizsgál meg.

Tegyük fel, hogy a gyorsítótár mérete (Z, L). Tegyük fel, hogy k darab számláló K' bájttal leírható. Valamint tegyük fel, hogy Z/L legalább akkora, hogy $2k$ darab számlálót, k darab tömbelemet, és egy ideiglenes elemet a cseréhez mindig a gyorsítótárban lehet tartani, függetlenül a elhelyezésüktől a memóriában. Ekkor:

- C kiszámításához k számlálót kell a gyorsítótárban tartani, miközben a A-t sorban bejárjuk. Ez legfeljebb $O(1+K'/L+N/L)$ memória műveletet igényel.

- S kiszámításához k számlálót kell összesíteni. Ez legfeljebb $O(1 + K'/L)$ memória műveletet igényel.

- T előállítása S másolásával, legfeljebb $O(1 + K'/L)$ memória művelet.

- Minden kulcs bejárása nem rosszabb, mint a teljes tömb bejárása, hiszen kulcsok szerint is sorba járjuk be, legfeljebb kihagyásokkal. A résztömbök végeit legfeljebb $O(k)$ memóriaművelettel a gyorsítótárba lehet hozni, és a köztes elemek bejárása legfeljebb $O(N/L)$ memóriaműveletet igényel. Feltettük, hogy a kulcsokhoz tartozó számlálók végig a gyorsítótárban vannak.

Tehát összesen a memóriaműveletek száma legfeljebb $O(1 + k + K'/L + N/L)$.

3.2. Mátrix szorzás

Legyen A egy $m \times n$ dimenziós mátrix, B egy $n \times p$ dimenziós mátrix, és legyen a feladat a $C = AB$ mátrix szorzat kiszámítása.

C definíció szerint $\forall i \in \{1, 2, ..., m\} : \forall j \in \{1, 2, ..., p\} : C_{i,j} = \sum_{k=1}^{n} : A_{i,k}B_{k,j}$.

A definíció szerinti implementáció három egymásba ágyazott ciklus, a külső az eredmény sorait járja be, a középső az eredmény oszlopait, és a belső ciklus a vektorszorzatot számolja ki. Ez a megoldás $\Theta(n^3)$ összeadást, szorzást, és memóriaműveletet igényel.

Tegyük fel, hogy a mátrixokat sorfolytonosan tároljuk. Tegyük fel, hogy $m = n = p$, és a gyorsítótár mérete (Z, L). Legyen n olyan nagy, hogy a gyorsítótár egy sora a mátrixok legfeljebb egy teljes sorát tárolja. Valamint legyen n legalább kétszer akkora, mint a gyorsítótár sorainak száma, Z/L.

Nézzük a definíció szerinti megoldás futtatásának azokat a memóriaeléréseit, ami a B mátrix elemeit olvassák. Ez n^2 alkalommal beolvassa a B mátrix egy-egy teljes oszlopát. Minden oszlop-beolvasás n értéket olvas be, és a feltevések szerint ehhez legalább n memóriasort kell beolvasni. Az oszlop beolvasásának az elején a gyorsítótár legfeljebb $n/2$ memóriasort tartalmaz. Így a teljes oszlop beolvasása legalább $n/2$ memóriaműveletet igényel. Tehát a definíció szerinti megoldás elegendően nagy mátrixok esetén legalább $n^3/2$ memóriaműveletet vált ki. Tehát a megoldás $\Theta(n^3)$ memóriaművelet bonyolultságú.

A definíció szerinti megoldás oszlopfolytonosan elrendezett mátrixot esetén is $\Theta(n^3)$ gyorsítótár bonyolultságú, a fenti érveléshez hasonlóan az A mátrix elérését vizsgálva.

3.2.1. Gyorsítótár tudatos

A mátrix szorzást block mátrixokkal is elvégezhetjük. Például legyen A és B egy-egy $2n \times 2n$ dimenziós mátrix, és osszuk fel mindkettőt 4 $n \times n$ dimenziós blokkokra.

$$A = \begin{pmatrix} A_{1,1} & A_{1,2} \\ A_{2,1} & A_{2,2} \end{pmatrix}$$
$$B = \begin{pmatrix} B_{1,1} & B_{1,2} \\ B_{2,1} & B_{2,2} \end{pmatrix}$$
$$C = \begin{pmatrix} C_{1,1} & C_{1,2} \\ C_{2,1} & C_{2,2} \end{pmatrix}$$

Ekkor $\forall i \in \{1,2\} : \forall j \in \{1,2\} : C_{i,j} = \sum_{k=1}^{2} A_{i,k} B_{k,j}$.

Legyen s a program egy paramétere. A mátrixok méretei legyenek s többszörösei, szükség esetén nullákkal kiegészítve a mátrixokat, és legyen $m = n = p$. A mátrixok legyen sorfolytonosan tárolva.

Tegyük fel, hogy a gyorsítótár magas[1], $Z = \Omega(L^2)$,

Legyen (Z, L) a gyorsítótár mérete, és s-t válasszuk a lehető legnagyobbnak úgy, hogy három darab $s \times s$ dimenziós mátrix befér a gyorsítótárba. Egy $s \times s$ mátrix $\Theta(s+s^2/L)$ gyorsítótár sorban fér el. A magas gyorsítótár feltétel miatt $s = \Theta(\sqrt{Z})$.

A blokkonkénti szorzást végezzük el a következőképpen:

- Nullázzuk a C mátrixot. Ez legfeljebb $O(n + n^2/L)$ memóriaművelettel jár. n sor van, és egy sor bejárása legfeljebb $O(1 + n/L)$ memóriaművelet.
- Járjuk be 3 ciklussal a rész-szorzatok blokk mátrixait. $i, j, k = 1, 2, ..., n/s$.
- Adjuk hozzá C részblokkjához a bemeneti részblokkok szorzatát.
  $C_{i,j} \leftarrow C_{i,j} + A_{i,k} B_{k,j}$.

A három mátrix gyorsítótára hozatala legfeljebb Z/L memóriaműveletet igényel, az s megválasztása miatt.

Összesen, nagy n-ekre, $\Theta(n + n^2/L + (n/\sqrt{Z})^3(Z/L)) = \Theta(n + n^2/L + n^3/L\sqrt{Z})$

3.2.2. Kettő-hatvány négyzetes mátrixok

Tegyük fel, hogy a mátrixok kettő-hatvány négyzetes mátrixok, $m = n = p = 2^k$. Legyenek a mátrixok sorfolytonosan elrendezve a memóriában.

A megoldásban rekurzívan osszuk a mátrixokat 4 felé. Legyen a rekurzió neve $Szorzat(A, B, C, n)$.

- Nullázzuk a C mátrixot. Ez legfeljebb $O(n + n^2/L)$ memóriaművelettel jár.
- Hívjuk meg a rekurzív függvényt, $Szorzat(A, B, C, n)$.
- A rekurzió alapesete az $n = 1$.
  $C_{1,1} \leftarrow C_{1,1} + A_{1,1}B_{1,1}$.
- Az általános eset $n > 1$. Osszuk mindhárom mátrixot négy-négy négyzetes blokkra.

$$A = \begin{pmatrix} A_{1,1} & A_{1,2} \\ A_{2,1} & A_{2,2} \end{pmatrix}$$
$$B = \begin{pmatrix} B_{1,1} & B_{1,2} \\ B_{2,1} & B_{2,2} \end{pmatrix}$$
$$C = \begin{pmatrix} C_{1,1} & C_{1,2} \\ C_{2,1} & C_{2,2} \end{pmatrix}$$

  Hívjuk meg nyolcszor a rekurziót.

  - $Szorzat(A_{1,1}, B_{1,1}, C_{1,1}, n/2)$.
  - $Szorzat(A_{1,2}, B_{2,1}, C_{1,1}, n/2)$.
  - $Szorzat(A_{1,1}, B_{1,2}, C_{1,2}, n/2)$.
  - $Szorzat(A_{1,2}, B_{2,2}, C_{1,2}, n/2)$.
  - $Szorzat(A_{2,1}, B_{1,1}, C_{2,1}, n/2)$.
  - $Szorzat(A_{2,2}, B_{2,1}, C_{2,1}, n/2)$.
  - $Szorzat(A_{2,1}, B_{1,2}, C_{2,2}, n/2)$.
  - $Szorzat(A_{2,2}, B_{2,2}, C_{2,2}, n/2)$.

A rekurzió műveletbonyolultságát leírja a következő rekurzív összefüggés.

$$T(1) = \Theta(1)$$
$$T(1) = 8T(n/2) + \Theta(1)$$

A mester tétel[9] felhasználásával $T(n) = \Theta(n^3)$.

Tegyük fel, hogy a gyorsítótár mérete (Z, L). Tegyük fel, hogy a gyorsítótár magas. Legyen t a legnagyobb olyan szám, hogy három darab 2^t méretű négyzetes mátrix elfér a gyorsítótárban. Legyen $s = 2^t$. Egy $s \times s$ mátrix $\Theta(s + s^2/L)$ gyorsítótár sorban fér el. A magas gyorsítótár feltétel miatt $s = \Theta(\sqrt{Z})$. A rekurzív fában az s méretű négyzetes mátrixok gyorsítótárba hozatala $\Theta((2^{k-t})^3 Z/L) = \Theta(n^3 Z/\sqrt{Z}^3 L) = \Theta(n^3/\sqrt{Z}L)$ memóriaműveletet igényel.

Az eredmény mátrix nullázásával együtt a memóriaműveletek száma a gyorsítótárral $\Theta(n + n^2/L + n^3/\sqrt{Z}L)$. Ez megegyezik a gyorsítótár tudatos megoldás gyorsítótár bonyolultságával, explicit gyorsítótár paraméter használata nélkül.

3.2.3. Gyorsítótár független

Általános méretű mátrixokra hasonló gyorsítótár független megoldást lehet adni, mint a kettő-hatvány négyzetes mátrixokra.

Legyen A egy $m \times n$ dimenziós mátrix, és B egy $n \times p$ dimenziós mátrix. Legyenek A, B, és C sorfolytonosan tárolva a memóriában. Legyen a rekurzió neve $Szorzat(A, B, C, m, n, p)$.

- Nullázuk a C mátrixot.
- Hívjuk meg a $Szorzat(A, B, C, m, n, p)$ mátrixot.
- A rekurzió alapesete az $m = n = p = 1$.
  $C_{1,1} \leftarrow C_{1,1} + A_{1,1}B_{1,1}$.
- Ha $m \geq \max(n, p)$, akkor legyen $m' = \lfloor m/2 \rfloor$, és osszuk fel A-t, és C-t,

  $$A = \begin{pmatrix} A_1 \\ A_2 \end{pmatrix}$$
  $$C = \begin{pmatrix} C_1 \\ C_2 \end{pmatrix}$$

  úgy, hogy A_1 és C_1 m' sorból áll. Hívjuk meg rekurzívan a függvényt:

  - $Szorzat(A_1, B, C_1, m', n, p)$
  - $Szorzat(A_2, B, C_2, m - m', n, p)$

- Ha $p \geq \max(m, n)$, akkor legyen $p' = \lfloor p/2 \rfloor$, és osszuk fel B-t, és C-t,

$$B = \begin{pmatrix} B_1 & B_2 \end{pmatrix}$$
$$C = \begin{pmatrix} C_1 & C_2 \end{pmatrix}$$

  úgy, hogy B_1 és C_1 p' oszlopból áll. Hívjuk meg rekurzívan a függvényt:

  - $Szorzat(A, B_1, C_1, m, n, p')$
  - $Szorzat(A, B_2, C_2, m, n, p - p')$

- Ha $n \geq \max(m, p)$, akkor legyen $n' = \lfloor n/2 \rfloor$, és osszuk fel A-t, és B-t,

$$A = \begin{pmatrix} A_1 & A_2 \end{pmatrix}$$
$$B = \begin{pmatrix} B_1 \\ B_2 \end{pmatrix}$$

  úgy, hogy A_1 n' oszlopból és B_1 n' sorból áll. Hívjuk meg rekurzívan a függvényt:

  - $Szorzat(A_1, B_1, C, m, n', p)$
  - $Szorzat(A_2, B_2, C, m, n - n', p)$

A megoldás időbonyolultsága $\Theta(mnp)$.

Tegyük fel, hogy a gyorsítótár mérete (Z, L). Tegyük fel, hogy a gyorsítótár magas. Ekkor a megoldás gyorsítótár bonyolultsága[1] $\Theta(m + n + p + (mn + mp + np)/L + mnp/LZ)$.

3.2.4. Strassen algoritmus

A Strassen algoritmus[10] a kettő-hatvány négyzetmátrixok szorzásához hasonlóan a mátrixokat négy-négy részre osztja, de az eredményt mindössze hét rekurzióval

számolja ki.

$$
\begin{aligned}
A &= \begin{pmatrix} A_{1,1} & A_{1,2} \\ A_{2,1} & A_{2,2} \end{pmatrix} \\
B &= \begin{pmatrix} B_{1,1} & B_{1,2} \\ B_{2,1} & B_{2,2} \end{pmatrix} \\
C &= \begin{pmatrix} C_{1,1} & C_{1,2} \\ C_{2,1} & C_{2,2} \end{pmatrix} \\
M_1 &= (A_{1,1} + A_{2,2})(B_{1,1} + B_{2,2}) \\
M_2 &= (A_{2,1} + A_{2,2})B_{1,1} \\
M_3 &= A_{1,1}(B_{1,2} - B_{2,2}) \\
M_4 &= A_{2,2}(B_{2,1} - B_{1,1}) \\
M_5 &= (A_{1,1} + A_{1,2})B_{2,2} \\
M_6 &= (A_{2,1} - A_{1,1})(B_{1,1} + B_{1,2}) \\
M_7 &= (A_{1,2} - A_{2,2})(B_{2,1} + B_{2,2}) \\
C_{1,1} &= M_1 + M_4 - M_5 + M_7 \\
C_{1,2} &= M_3 + M_5 \\
C_{2,1} &= M_2 + M_4 \\
C_{2,2} &= M_1 - M_2 + M_3 + M_6
\end{aligned}
$$

A rekurzív szorzások az M mátrixok kiszámításához szükségesek.

A Strassen algoritmus időbonyolultsága[10] $\Theta(n^{\log_2 7})$.

Legyen a gyorsítótár mérete (Z, L). Ekkor a Strassen algoritmus gyorsítótár bonyolultsága[1] $\Theta(n + n^2/L + n^{\log_2 7}/L\sqrt{Z})$.

3.2.5. Z-sorrend

Legyen A egy kettő-hatvány méretű négyzetes mátrix, és osszuk fel egyenlő méretű blokkokra.

$$
A = \begin{pmatrix} A_{1,1} & A_{1,2} \\ A_{2,1} & A_{2,2} \end{pmatrix}
$$

A memóriában úgy helyezzük el az A mátrixot, hogy először $A_{1,1}$ összes értékét tároljuk el, majd $A_{1,2}$-t, $A_{2,1}$-et, végül $A_{2,2}$-t. A blokkokat rekurzívan osszuk fel, ha nagyobbak mint egy, és azokat is ilyen sorrendben tároljuk. Például egy 4×4

dimenziós mátrix elemeinek sorrendje a memóriában:

$$A = \begin{pmatrix} 1 & 2 & 5 & 6 \\ 3 & 4 & 7 & 8 \\ 9 & 10 & 13 & 14 \\ 11 & 12 & 15 & 16 \end{pmatrix}$$

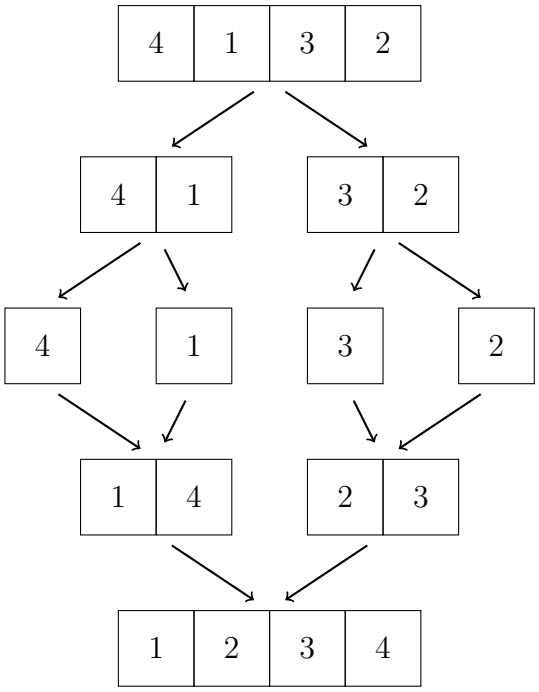
Ebben az elrendezésben a magas gyorsítótár feltételezés helyett elég a gyengébb feltétel is, hogy a a gyorsítótár sorainak száma egy elegendően nagy konstans.[1] Ha egy $s \times s$ méretű részmátrix $O(1)$ gyorsítótár sorban tárolható, akkor elég ha a gyorsítótárnak $3O(1) = O(1)$ sora van, hogy egy $s \times s$ méretű részfeladatot a gyorsítótárba egyszer beolvasva megoldjunk.

3.3. Rendezés

3.3.1. Összefésüléses rendezés

Az összefésüléses rendezés egy stabil rendezés. Egy rendezés stabil, ha egyenlő elemek sorrendje a bemeneti és a kimeneti listában megegyezik. Az összefésüléses rendezés összefésülés lépése két rendezett listából előállít egy listát, ami a két lista elemeit tartalmazza, és szintén rendezett. Tegyük fel, hogy a rendezendő lista $L[1 : n]$, ahol n a lista hossza. A rekurzív algoritmus a következő.

- Az $n \leq 1$ a rekurzió alapesete. Az üres és az egyelemű lista rendezett.
- Ha $n > 1$, akkor legyen $k = \lfloor n/2 \rfloor$.
- Rendezzük rekurzívan $L[1 : k]$-t, és $L[k + 1 : n]$-et.
- Fésüljük össze a már rendezett $L[1 : k]$-t, és $L[k + 1 : n]$-et.
  - Legyen az eredménylista üres.
  - Legyen a bal és a jobb lista a rendezett $L[1 : k]$, és $L[k + 1 : n]$.
  - Amíg se a bal, se a jobb lista nem üres, addig hasonlítsuk össze, hogy a két lista első elemét, és válasszuk azt a listát, ahol az első elem kisebb. Ha a két lista első eleme megegyezik, akkor válasszuk a bal listát. Távolítsuk el a kiválasztott lista első elemét, és fűzzük az eredménylista végére.
  - Fűzzük a maradék bal listát az eredménylista végére.
  - Fűzzük a maradék jobb listát az eredménylista végére.



3.1. ábra. Az összefésüléses rendezés futása

Tegyük fel, hogy a rendezendő lista mérete n. Ekkor az összefésüléses rendezés időbonyolultsága $\Theta(n \log n)$. Tegyük fel, hogy a gyorsítótár mérete (Z/L). Ekkor az összefésüléses rendezés gyorsítótár-bonyolultsága $\Theta(\frac{n}{L} \log \frac{n}{Z})$ [1].

Az időbonyolultságot analizálhatjuk a mester-tétel[9] segítségével. Egy n elemű lista szétválasztása, és összefésülése $\Theta(n)$ műveletet igényel. Legyen $T(n)$ az összefésüléses rendezés időbonyolultsága. Ez megfelel a következő rekurziónak.

$$T(n) = \begin{cases} O(1) & \text{ha } n = 1 \\ 2T(n/2) + O(n) & \text{különben} \end{cases}$$

Ekkor a mester-tétel szerint $T(n) = \Theta(n \log n)$.

Az időbonyolultság megbecsléséhez a rekurzív fát is vizsgálhatjuk.

- A rekurzív fának $\Theta(\log n)$ szintje van.
- Minden szinten a listák elemszámainak összege legfeljebb n, és szinte minden szinten egyenlő n-nel.
- Egy adott szinten az összes összefésülés legfeljebb $O(n)$ műveletet igényel, és szinte minden szinten $\Theta(n)$ műveletet.
- Legfeljebb a legalsó szint nem tartalmaz összesen n elemet.
- Összes művelet száma így $\Theta(n \log n)$.

A gyorsítótár-bonyolultság analíziséhez is használjuk a rekurzív fát.

- Ha egy n elemű részlista, és a segédstruktúrák teljes egészében elférnek a Z méretű gyorsítótárban, akkor a rekurzió ezen része legfeljebb $O(Z)$ memóriaműveletet végez.
- A rekurzív fának $\Theta(\log \frac{n}{Z})$ szintje van, mert az időbonyolultság rekurzív fájának alsó $\Theta(\log Z)$ szintje nem jár további memóriaművelettel.
- Egy adott szinten sorban kell bejárni tömböket. Egy k elemű lista összefésülése $O(\frac{k}{L} + 1)$ művelettel jár. Ha a teljes szinten m listát kell összefésülni, akkor azt összesen $O(\frac{n}{Z} + m)$ memóriaművelettel jár. A d-edik szinten $m \leq 2^d$.
- Összesen a domináns tag tehát $\Theta(\frac{n}{L} \log \frac{n}{Z})$.
- A töredék sorok beolvasása miatt műveletek száma legfeljebb

$$\sum_{i=0}^{O(\log \frac{n}{Z})} 2^i = O\left(2^{O(\log \frac{n}{Z})}\right) = O\left(\frac{n}{Z}\right)$$

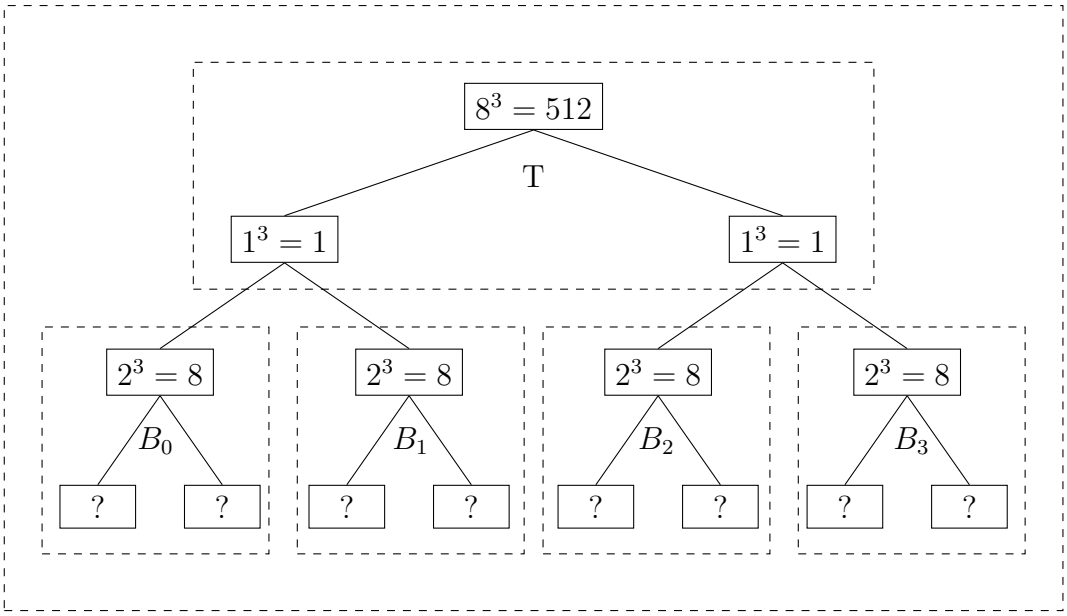
  Ez aszimptotikusan kisebb, mint $O(\frac{n}{L} \log \frac{n}{Z})$.

3.3.2. Tölcsér rendezés

A lusta tölcsér rendezés[16] egy összefésülésen alapuló rekurzív rendezés. A rekurzív algoritmus hasonló az összefésüléses rendezéshez, a fő különbség a többirányú összefésülés. Legyen n a rendezendő tömb mérete.

- Az $n \leq 1$ a rekurzió alapesete, az üres és egyelemű tömb rendezett.
- Ha $n > 1$:
  - Legyen $k = \lceil n^{1/3} \rceil$.
  - A tömböt osszuk fel k darab közel egyenlő hosszú résztömbre.
  - Rendezzük a k darab résztömböt rekurzívan.
  - Egy k-összefésülővel fésüljük össze a k darab résztömböt. A k összefésülő kimeneti pufferének mérete legyen n

Egy k-összefésülő k rendezett tömböt fésül össze egy rendezett tömbbé. A k-összefésülőket kettő-hatvány k-ra adjuk meg. Ha k nem kettő-hatvány, akkor üres tömbökkel ki lehet egészíteni az összefésülendő tömbök listáját kettő-hatvány számosságúra. Legyen $h = \log_2 k$. Ekkor egy k-összefésülő egy h magasságú teljes bináris fa. A fa levelei a k darab bemeneti puffer. A fa belső csúcsai egy kimeneti pufferből állnak, és egy 2-összefésülőből. A 2-összefésülő ugyanaz, mint az összefésüléses rendezésben. A 2-összefésülő bemenetei a csúcs két gyerekének kimeneti pufferei. A k-összefésülő kimeneti puffere a gyökér kimeneti puffere.



3.2. ábra. Egy 8-összefésülő elrendezése a memóriában, és a kimenő pufferek méretei

A pufferek FIFO sorok, a következő tulajdonságokkal.

- Minden puffernek van egy előre meghatározott maximális mérete.
- Támogatja a szokásos sor műveleteket:
  - Tele van?
  - Üres?
  - Elem hozzáadása a sor végéhez.
  - Elem elvétele a sor elejéről.
  - Betekintés a sor első elemébe.
- Elem kivétele után addig nem adható új elem a sorhoz, amíg az ki nem ürül teljesen.

Egy 2-összefésülő eljárása a Feltölt. Ez addig futtatja a két bemenet összefűzését, amíg a 2-összefésülő kimenete meg nem telik, vagy elfogynak az elemek a bemeneti pufferekben. Ha egy bemeneti pufferben az elemek elfogynak, akkor az rekurzívan meghívja a kiürült pufferhez tartozó Feltölt eljárást. Ha ezek után is üres a bemeneti puffer, akkor az megjelöli ténylegesen üresnek.

Egy k-összefésülő pufferméreteit és az elrendezését a memóriában rekurzívan adjuk meg. Tegyük fel, hogy a k-összefésülő kimeneti pufferének mérete n kell legyen. Tegyük fel, hogy $k \geq 2$, és $h = \log_2 k$. h egész szám, mivel k kettőhatvány. Legyen $d = \lfloor h/2 \rfloor$. Legyen T az összefésülőfa azon csúcsok által indukált részfája, amiknek a távolság a gyökértől legfeljebb d. Legyen $m = 2^{d+1}$. Legyenek

$B_1, B_2, ..., B_m$ az összefésülőfa részfái, amiknek a gyökerei azon csúcsok, amiknek a távolsága a gyökértől $d + 1$. Egy ilyen felosztás látható a 3.2 ábrán.

Ezek után a memóriában sorfolytonosan helyezzük el először a T-hez tartozó 2^d-összefésülőt, aminek a kimeneti pufferének mérete n. Majd sorban minden B_i-re rendezzünk el egy k/m-összefésülőt, aminek a kimeneti pufferének mérete legyen $(k/m)^3$.

Ezek után k darab rendezett tömböt, amiknek összesen az elemszáma n, összefésülhetünk egy n méretű kimeneti pufferrel rendelkező k-összefésülő felépítésével, majd a Feltölt eljárás egyszeri meghívásával.

Megmutatható[16], hogy egy k-összefésülő tárigénye a kimeneti puffer nélkül legfeljebb $O(k^2)$. Tegyük fel, hogy a rendezendő elemek száma n. Tegyük fel, hogy a gyorsítótár mérete (Z/L), és a gyorsítótár magas, azaz $Z = \Omega(L^2)$.

Szintén megmutatható, hogy a tölcsér rendezés gyorsítótár-bonyolultsága $\Theta(\frac{n}{L} \log_Z n)$. Ez $\ln Z$ faktorral jobb, mint az összefésüléses rendezés. Ha egy k-összefésülő teljesen egészében elfér a gyorsítótárban, valamint az általa használt bemeneti pufferek mindegyikéből is elfér egy-egy block a gyorsítótárban, és a bemeneti pufferek tele vannak, akkor az összefésülő kimeneti pufferének feltöltéséhez elég az összefésülőt egyszer beolvasni a memóriába, és a kimeneti puffert, valamint a bemeneti puffereket egyszer bejárni. Ezek a k-összefésülők a gyorsítótár szempontjából úgy viselkednek, mintha egy atomi összefésülő lenne k bemeneti pufferrel.

3.4. Edénysor szita

Az edénysor szität a szakdolgozatomban írtam le. Ez egy eratoszthenészi rekurzívan szegmentált szita.

Az eratoszthenészi sziták prímek többszöröseit jelölik meg egy adott intervallumon, és egy-egy prím többszöröseinek megjelöléseit a többi prímtől függetlenül végzik. A szita alkalmas prímek keresésére is, és az intervallum számainak prímfaktorizációjára is.

Egy szegmentált szita a teljes intervallumot kisebb intervallumokra osztja, ezek a szegmensek, és a szegmensekben külön-külön végzi el a megjelöléseket. A szegmensek átfedés nélkül lefedik a teljes intervallumot. A szegmens méretet úgy választják meg, hogy szitáláshoz használt prímek és egy szegmens a gyorsítótárban elférjen. Ha a szegmensek mérete a program bemeneti paramétere, akkor a szita gyorsítótár tudatos.

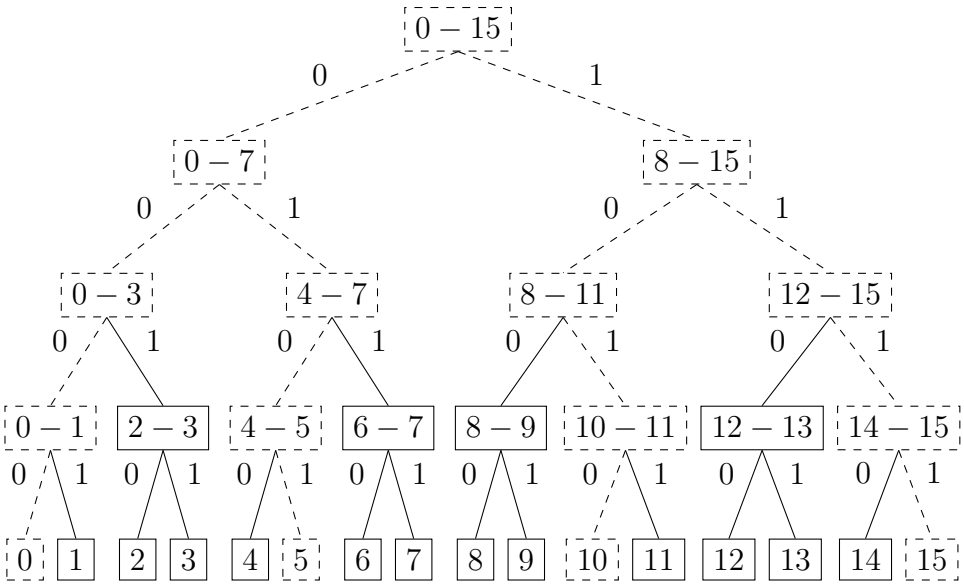
A rekurzív edényszita egy a^h hosszú intervallumot szitál, ahol a és h természetes számok, $a \geq 2$. Ha $h > 0$, akkor az intervallumot a darab ugyanolyan hosszú részre osztja, és ezeket rekurzívan szitálja. A prímek listáját előszűri, hogy egy szegmensbe csak olyan prímeket vizsgáljon, amiknek ténylegesen van többszöröse a szegmensben.

3.4.1. A rekurzív fa

Vegyük a $[u, u+a^h)$ intervallumot, u természetes szám. és egy p prímet, aminek van többszöröse az intervallumban. Tegyük fel, hogy $h > 0$. Ekkor az i. részintervallum $[u + ia^{h-1}, u + (i+1)a^{h-1})$, minden $i \in 0, ..., a-1$-re.

- Ha $p \geq a^h$, akkor p-nek pontosan egy részintervallumban van többszöröse.
- Ha $a^{h-1} < p < a^h$, akkor p-nek legalább egy részintervallumban van többszöröse, de nem feltétlen mindegyikben.
- Ha $p \leq a^{h-1}$, akkor p-nek minden részintervallumban van többszöröse.

Példának nézzük meg, hogy az 5 milyen utat jár be a rekurzív fában a $[0, 2^4)$ intervallum szitálásakor. Az ábrán a prím 0-t is bejárja, ami prímek keresésekor nem következik be.



3.3. ábra. Az 5 többszörösei a $[0, 2^4)$ intervallumhoz tartozó rekurzív fában.

3.4.2. Rekurzív szita

Járjuk be a $[u, u + a^h)$ intervallumhoz tartozó rekurzív fát. A prímek listáján legyen minden olyan prím, ami kisebb u-nál. Minden levelet jelöljünk meg azokkal a prímekkel, amik a levélhez tartozó számot osztják. Ha egy levelet egyetlen prím sem oszt a listán, akkor az a szám prím, jelöljük meg magával, és adjuk a listához. A fa bejárásánál minden csúcsban előfeltétel, hogy a prímek listáján minden prím legalább egy levélhez tartozó számot oszt.

Egy belső csúcsban a gyerekeket sorban bejárjuk, és a prímek listáját előszűrjük, hogy megfeleljen a gyerekekre vonatkozó előfeltételnek. Azt az előszűrést kétféleképpen is elvégezhessük.

- A prímek listájában csak a prímeket tároljuk, és egy részintervallum végpontjainak ismeretében maradékos osztással eldöntjük, hogy van-e egy prímnek többszöröse. Ekkor a memóriaigényt számítási igényre cseréljük.

- A prímek listájában minden prím mellé eltároljuk, hogy melyik szám a következő többszöröse, és ezt hasonlítjuk össze az éppen következő részintervallummal. Ekkor a számítási igény memória igényre cseréljük.

A $[0, a^h)$ intervallum prím-faktorizációját a RekurzívSzita$(a, h, a^h, 0, \emptyset)$ hívással lehet elvégezni. A hívás után az intervallum egy száma prím, ha nincs megjelölve, és összetett, ha meg van jelölve, és az összes prímosztója megjelölte.

Prímek keresését a $[0, a^h)$ intervallumban a RekurzívSzita$(a, h, \left\lfloor \sqrt{a^h} \right\rfloor, 0, \emptyset)$ hívás lehet elvégezni. Az eredménylista tartalmazza a prímeket. Ebben az esetben a levelek megjelölése nem szükséges.

3.4.3. Műveletigény gyorsítótár nélkül

Tegyük fel, hogy a $[0, a^h)$ intervallumot szitáljuk. Legyen $n = a^h$, és m a minket érdeklő prímek felső korlátja. Legyen $p \leq m$ egy prím.

Nézzük a rekurzív fában az összes a^k hosszú részintervallumot, $k \leq h$.

- Ha $p > a^k$, akkor p-nek legfeljebb egy többszöröse van egy adott részintervallumban, és összesen $\lfloor n/p \rfloor$ többszöröse van, és ennyi a^k hosszú részintervallumot jár be.

- Ha $p \leq a^k$, akkor p-nek minden részintervallumban van többszöröse.

Mivel a-nak csak egész hatványai lehetnek intervallumok hosszai, ezért

$$p \leq a^k \Leftrightarrow log_a p \leq k \Leftrightarrow \lceil log_a p \leq k \rceil$$

Algoritmus 3.2 Az $[u, u + a^h)$ intervallum szitálása

Require: a: a számrendszer alapja
Require: h: a fa magassága
Require: m: a keresett prímek felső korlátja
Require: u: a az intervallum kezdőpontja
Require: P: lista, azok a p prímszámok, hogy létezik p-nek többszöröse az $[u, u + a^h)$ intervallumban
Ensure: P: P kiegészítve az intervallumban talált prímekkel

```
function RekurzívSzita(a, h, m, u, P) → P
    if h = 0 then
        if P üres és u ≥ 2 then                ▷ u egy újonnan megtalált prím
            if u ≤ m then
                P ← P ∪ {u}
            end if
        end if
        jelöljük meg u-t P minden elemével
    else
        legyenek a részintervallumok I_i = [u + ia^{h−1}, u + (i + 1)a^{h−1}, i ∈ [0, a)
        Q legyen listák egy a elemű tömbje, és legyen minden lista üres
        for p ∈ P do
            Legyen i a legkisebb olyan szám, hogy p-nek van többszöröse I_i-ben
            Q[i] ← Q[i] ∪ {p}
            P ← P \ {p}
        end for
        for i = 0, ..., a − 1 do
            Q[i] ← RekurzívSzita(a, h − 1, m, u + ia^{h−1}, Q[i])
            for p ∈ Q[i] do
                if p-nek van többszöröse I_{i+1}, ..., I_{a−1}-ben then
                    legyen j a legkisebb olyan szám,
                    hogy p-nek van többszöröse I_{i+1}, ..., I_{a−1}-ben
                    Q[j] ← Q[j] ∪ {p}
                else
                    P ← P ∪ {p}
                end if
                Q[i] ← Q[i] \ {p}
            end for
        end for
    end if
    return P
end function
```

Így a rekurzív fában p legfeljebb a felső $h - \lceil log_a p \rceil + 1$ szinten jár be minden csúcsot. Az alsó $\lceil log_a p \rceil$ szinten szintenként $\lfloor n/p \rfloor$ csúcsot jár be. Legyen C_p azon csúcsok száma, amit p bejár.

$$C_p \leq \sum_{k=\lceil \log_a p \rceil}^{h} a^{h-k} + \sum_{k=0}^{\lceil \log_a p \rceil - 1} \left\lfloor \frac{n}{p} \right\rfloor$$

Vizsgáljuk az összeg baloldalát:

$$\begin{aligned}
\sum_{k=\lceil \log_a p \rceil}^{h} a^{h-k} &= \sum_{k=0}^{h-\lceil \log_a p \rceil} a^k \\
&= \frac{a^{h-\lceil \log_a p \rceil + 1} - 1}{a-1} \\
&< \frac{a^{h-\lceil \log_a p \rceil + 1}}{a-1} \\
&\leq \frac{2a^{h-\lceil \log_a p \rceil + 1}}{a} \\
&= \frac{2a^h}{a^{\lceil \log_a p \rceil}} \\
&= \frac{2n}{a^{\lceil \log_a p \rceil}} \\
&\leq \frac{2n}{a^{\log_a p}} \\
&= \frac{2n}{p}
\end{aligned}$$

Vizsgáljuk az összeg jobboldalát:

$$\begin{aligned}
\sum_{k=0}^{\lceil \log_a p \rceil - 1} \left\lfloor \frac{n}{p} \right\rfloor &= \lceil \log_a p \rceil \left\lfloor \frac{n}{p} \right\rfloor \\
&< (\log_a p + 1) \left\lfloor \frac{n}{p} \right\rfloor \\
&\leq (\log_a p + 1) \frac{n}{p} \\
&= \left(\frac{\ln p}{\ln a} + 1 \right) \frac{n}{p} \\
&= \frac{n \ln p}{p \ln a} + \frac{n}{p}
\end{aligned}$$

Az összegre vonatkozó két becslést egyesítve:

$$C_p < \frac{2n}{p} + \frac{n \ln p}{p \ln a} + \frac{n}{p} = \frac{3n}{p} + \frac{n \ln p}{p \ln a}$$

Az összes prím által meglátogatott csúcsok számának becsléséhez felhasználjuk

Mertens két tételét.[11]

$$\sum_{p \le n} \frac{\ln p}{p} = \ln n + O(1)$$
$$\sum_{p \le n} \frac{1}{p} = \ln \ln n + O(1)$$

Így az összes prím által meglátogatott csúcsok száma multiplicitással:

$$\begin{aligned}
C &= \sum_{p \le m} C_p \\
&< \sum_{p \le m} \left(\frac{3n}{p} + \frac{n \ln p}{p \ln a} \right) \\
&= \sum_{p \le m} \frac{3n}{p} + \sum_{p \le m} \frac{n \ln p}{p \ln a} \\
&= 3n \sum_{p \le m} \frac{1}{p} + \frac{n}{\ln a} \sum_{p \le m} \frac{\ln p}{p} \\
&= 3n(\ln \ln m + O(1)) + \frac{n}{\ln a}(\ln m + O(1)) \\
&= \frac{n \ln m}{\ln a} + 3n \ln \ln m + O(n)
\end{aligned}$$

Tegyük fel, hogy $m = n^q$, $0 < q \le 1$ konstansra. Tipikusan $q \in \{\frac{1}{2}, 1\}$. Ekkor:

$$\begin{aligned}
C &< \frac{n \ln n^q}{\ln a} + 3n \ln \ln n^q + O(n) \\
&= \frac{nq \ln n}{\ln a} + 3n \ln (q \ln n) + O(n) \\
&= \frac{nq \ln n}{\ln a} + 3n \ln q + 3n \ln \ln n + O(n) \\
&= O\left(\frac{n \ln n}{\ln a} + n \ln \ln n \right)
\end{aligned}$$

A RekurzívSzita program egy adott belső csúcsában egy adott prímmel többször is foglalkozik.

- Kiosztja legalább egy részintervallumra. Intervallumonként $O(1)$ időben.
- Összegyűjti a visszatérési értékbe. Egyszer $O(1)$ időben.

Valamint vannak a-tól függő inicializálási lépések, $O(a)$ időben. Egy levél csúcsban minden prímmel $O(1)$ időt foglalkozik.

Az összesítéshez számoljuk el a a részintervallumra kiosztás költségét a részintervallum csúcsában. Így

- Egy levélben minden prímmel $O(1)$ időt foglalkozik.
- Egy belső csúcsban $O(a)$ időt tölt karbantartással, és prímenként $O(1)$ időt azzal, hogy rá osztották, vagy visszatér vele.

Tehát a teljes időigény

$$\begin{aligned}
T &= \sum_{k=1}^{h} a^{h-k} O(a) + CO(1) \\
&= \sum_{k=0}^{h-1} a^{k} O(a) + CO(1) \\
&= \frac{a^h - 1}{a - 1} O(a) + CO(1) \\
&= O(a^h) + CO(1) \\
&= O(n) + CO(1) \\
&= O(n) + O\left(\frac{n \ln n}{\ln a} + n \ln \ln n\right) \\
&= O\left(\frac{n \ln n}{\ln a} + n \ln \ln n\right)
\end{aligned}$$

3.4.4. A műveletigény alsó korlátja

Legyen a feladat a $[2, n)$ intervallumban minden szám prímfaktorizációja, $n = a^h$. Minden szám mellé írjuk fel az összes prímosztóját, egy a elemű ábécé felett. Minden ilyen feljegyzést az összes többitől függetlenül végezzünk el, azaz egy egy bejegyzés önmagában legyen értelmezhető.

Legyen $f_p = \left\lfloor \frac{n}{p} \right\rfloor$ a p prímet leíró szó frekvenciája a megoldásban. Legyen $F = \sum_{p \leq n} f_p$ az összesen leírt szavak száma. Mertens tétele[11] alapján alsó becsléseket

adunk F-re.

$$\begin{aligned}
F &= \sum_{p \le n} f_p \\
&= \sum_{p \le n} \left\lfloor \frac{n}{p} \right\rfloor \\
&> \sum_{p \le n} \left(\frac{n}{p} - 1 \right) \\
&= \sum_{p \le n} \frac{n}{p} - \sum_{p \le n} 1 \\
&= \sum_{p \le n} \frac{n}{p} - \Pi(n) \\
&= \sum_{p \le n} \frac{n}{p} + \Theta\left(\frac{n}{\ln n} \right) \\
&= n \sum_{p \le n} \frac{1}{p} + \Theta\left(\frac{n}{\ln n} \right) \\
&= n \left(\ln \ln n + O(1) \right) + \Theta\left(\frac{n}{\ln n} \right) \\
&= n \ln \ln n + O(n)
\end{aligned}$$

Tehát a megoldás legalább $\Omega(n \ln \ln n)$ szó leírásával jár.

Adjunk F-re egy durvább alsó becslést. Válasszuk N_1-et olyan nagyra, hogy minden $n \ge N_1$-re:

$$F > n \ln \ln n + O(n) \ge n$$

Mertens tétele[11] alapján

$$\sum_{p \ge n} \frac{\ln p}{p} = \ln n + R(n), \text{ ahol } |R(n)| < 2$$

Ez alapján válasszuk meg úgy N_2-t, hogy minden $n \ge N_2$-re

$$\sum_{p \ge n} \frac{\ln p}{p} = \frac{\ln n}{2}$$

Tegyük fel, hogy $n \ge \max(N_1, N_2)$.

Legyen P_p a p prím relatív gyakorisága. P_p-re egy felső becslést adunk.

$$P_p = \frac{f_p}{F} = \frac{\left\lfloor \frac{n}{p} \right\rfloor}{F} \le \frac{\frac{n}{p}}{F} < \frac{\frac{n}{p}}{n} = \frac{1}{p}$$

Tudjuk[13], hogy egy a elemű ábécé feletti optimális prefix kód esetén egy P

relatív gyakoriságú szó hossza legalább $-\log_a P$. Legyen O egy optimális kód a feladat megoldására. Legyen l_p a p prímhez tartozó kódszó hossza O-ban. l_p-re alsó becslést adunk.

$$l_p \geq -\log_a P_p > -\log_a \frac{1}{p} = \log_a p$$

Legyen L az O kóddal leírt megoldás hossza. L-ra alsó becslést adunk.

$$\begin{aligned}
L &\geq \sum_{p \leq n} f_p l_p \\
&> \sum_{p \leq n} f_p \log_a p \\
&= \frac{1}{\ln a} \sum_{p \leq n} f_p \ln p \\
&= \frac{1}{\ln a} \sum_{p \leq n} \left\lfloor \frac{n}{p} \right\rfloor \ln p \\
&> \frac{1}{\ln a} \sum_{p \leq n} \left(\frac{n}{p} - 1 \right) \ln p \\
&\geq \frac{1}{\ln a} \sum_{p \leq \frac{n}{2}} \left(\frac{n}{p} - 1 \right) \ln p \\
&\geq \frac{1}{\ln a} \sum_{p \leq \frac{n}{2}} \frac{n}{2p} \ln p \\
&= \frac{n}{2 \ln a} \sum_{p \leq \frac{n}{2}} \frac{\ln p}{p} \\
&\geq \frac{n \ln \frac{n}{2}}{4 \ln a} \\
&= \frac{n(\ln n - \ln 2)}{4 \ln a} \\
&= \frac{n \ln n}{4 \ln a} - \frac{n \ln 2}{4 \ln a} \\
&= \Omega \left(\frac{n \ln n}{\ln a} \right)
\end{aligned}$$

A prímfaktorizálás fenti definíciója nem tökéletesen egyezik a rekurzív szitával. A faktorizálásban megköveteltük, hogy egy véges ábécé felett kell megadni a prímfaktorokat. A rekurzív szitában a számokat egy véges ábécé felett jártuk be, de a megjelöléshez használt prímek listájában feltettük, hogy a prímek n-től függetlenül $O(1)$ memóriával leírhatóak, és $O(1)$ műveletigénnyel manipulálhatóak.

3.4.5. Gyorsítótár független edény szita

Megadjuk a RekurzívSzita egy gyorsítótár független implementációját, $a = 2$-re.

A RekurzívSzita programban a lista implementációját nem adtuk meg. A lista legyen a memória egy összefüggő része. A memória legyen elég nagy, hogy a futás során az összes prímet tárolni tudja. A memóriát a futás során 3 részre osztjuk.

- A lista elején azok a prímek vannak, amiknek az adott csúcsban nincs többszörösük. Ez a szakasz tovább fel van osztva, a rekurzív hívások vermének minden szintjéhez tartozik egy egybefüggő szakasz, ahol azok a prímek vannak, amiknek a híváshoz tartozó csúcsban van többszörösük, de a gyerek-hívás csúcsában nincs többszörösük. A lista legelején a legelső híváshoz tartozó prímek vannak.
- a lista közepén azok a prímek vannak, amiknek az adott csúcsban van többszörösük.
- a lista végén a nem használt memóriahelyek vannak.

A RekurzívSzita hívásnál bemenő paraméter a lista három pontja.

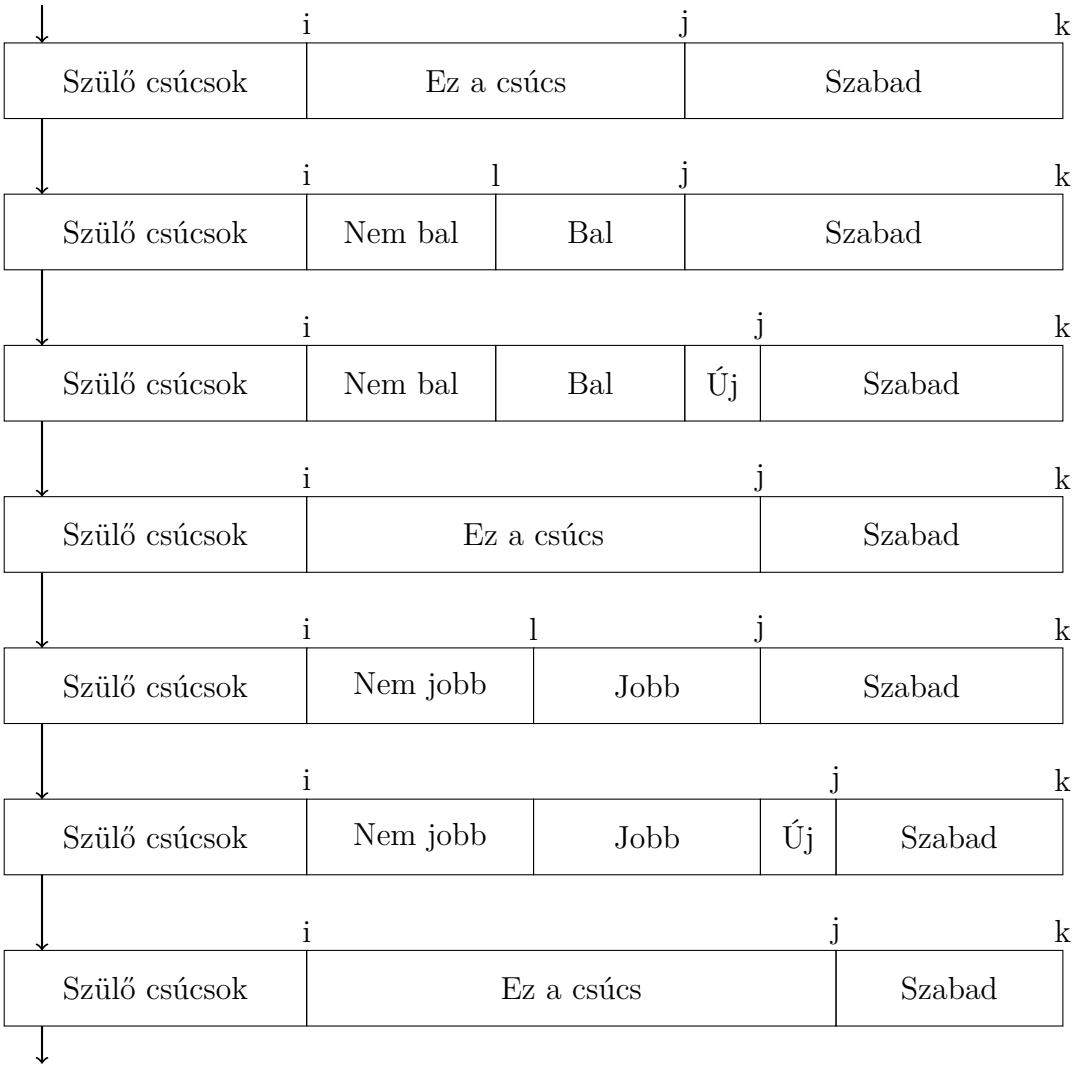
- A cím, ahonnan kezdve azok a prímek vannak a listában, amiknek ebben a csúcsban van többszöröse.
- A cím, ameddig a többszörössel rendelkező prímek tartanak, és a szabad helyek kezdődnek.
- Opcionálisan, a cím, amíg a szabad helyek tartanak. Ezt lehet használni annak detektálására, hogy elfogyott a hely a listában.

A kimenő paraméter a cím a listában, ameddig a csúcsban többszörössel rendelkező prímek tartanak. Ez lehet nagyobb a bemenő paraméternél, ha a szita új prímet talált.

Egy belső csúcsban a listát két particionálással elő lehet készíteni a rekurzív hívásra.

- Tegyük fel, hogy L a lista, és a három cím a listában $i \leq j \leq k$. $A[i : j]$ tartalmazza a prímeket, amiknek van többszöröse a csúcsban. $A[j : k]$ a szabad helyek.
- Particionáljuk az $A[i : j]$ listát, úgy, hogy azokat a prímeket, amiknek nincs a bal gyerekben többszörösük, azokat a lista elejére helyezzük. A partíciókat elválasztó cím legyen l. Ekkor az $A[l : j]$ tartalmazza a bal gyerekben többszörössel rendelkező prímeket, és $A[j : k]$ a szabad elemek.
- Hívjuk meg rekurzívan a szitát a bal gyerekre, a három cím legyen l, j, és k.
- Az eredmény címmel írjuk felül j-t. Ezzel bekerül a hívás ezen szintjén is a listába minden új prím.

- Particionáljuk az $A[i : j]$ listát, úgy, hogy azokat a prímeket, amiknek nincs a jobb gyerekben többszörösük, azokat a lista elejére helyezzük. A partíciókat elválasztó cím legyen l. Ekkor az $A[l : j]$ tartalmazza a jobb gyerekben többszörössel rendelkező prímeket, és $A[j : k]$ a szabad elemek.
- Hívjuk meg rekurzívan a szitát a jobb gyerekre, a három cím legyen l, j, és k.
- Az eredmény címmel írjuk felül j-t. Ezzel bekerül a hívás ezen szintjén is a listába minden új prím.
- Adjuk vissza j-t.



3.4. ábra. Gyorsítótár független particionálás egy csúcsban

Tegyük fel, hogy a gyorsítótár mérete (Z, L). Valamint tegyük fel, hogy egy csúcsban a prímek listájának mérete P a kimenetben. Ekkor a particionálások a csúcsban legfeljebb $O(1 + P/L)$ memóriaművelettel elvégezhetőek.

3.4.6. Gyorsítótár tudatos edény szita

Láncolt listával megadható a RekurzívSzita egy gyorsítótár tudatos implementációja.

- A lista minden elem legyen a memória egy rögzített hosszúságú egybefüggő része.
- Egy listaelem első tagja legyen egy mutató a következő elemre.
- Ha a listaelemnek egynél több prím is lehet az adattagja, akkor egy listaelemnek a második tagja legyen az elemben kitöltött prím adattagok száma.
- Egy listaelem maradék adattagja legyenek a prímek.
- Egy listaelemben a kitöltött prím adattagok megelőzik a ki nem töltötteket.
- Egy listában csak az első listaelem lehet nem teljesen kitöltött.
- A RekurzívSzita-ban a listák bejárásával egyidőben a lista elemeit töröljük. A láncolt lista a fejelem adattagjainak ismételt feldolgozásával, és a fejeleme felszabadításával feldolgozható.
- A szabad listaelemeket egy segédlistában tároljuk, veremként. Tehát listaelem allokálásnál a szabad elemek közül az időben legközelebb felszabadítottat választjuk ki.

Tegyük fel, hogy egy listaelem legfeljebb E prímet képes tárolni. Tegyük fel, hogy a gyorsítótár mérete (Z, L). Egy listaelem feldolgozása az érintett memóriaszakasz egyszeri végigolvasásával, legfeljebb $O(1 + E/L)$ memóriaművelettel elvégezhető.

Tegyük fel, hogy egy listaelem mérete a memóriában nagyobb, mint L, és hogy egy listaelemben a prímek memóriamérete legalább a fele a teljes listaelem méretének. Valamint tegyük fel, hogy egy csúcsban a prímek listájának mérete P a kimenetben. Ekkor a prímek listája a csúcsban legfeljebb $O(1 + P/E)$ listaelemet tartalmazhat, és ezek legfeljebb

$$O(1 + P/E)O(1 + E/L) = O(1 + E/L + P/E + P/L) = O(1 + P/L)$$

memóriaművelettel végigolvashatóak.

3.4.7. Műveletigény gyorsítótárral

Tegyük fel, hogy a gyorsítótár mérete (Z, L). Tegyük fel, hogy egy adott, a^k hosszú intervallumhoz tartozó, csúcsban a gyorsítótárban elfér a prímek teljes listája, és

minden szükséges segédstruktúra. Ekkor a RekurzívSzita a csúcsot és minden leszármazottját legfeljebb Z/L memóriaművelettel feldolgozza.

Az elemzéshez tegyük fel, hogy a prímek többszörösei minden adott szinthez tartozó csúcs között teljesen egyenletesen oszlanak el, akár tört súllyal is. Ez alapján tegyük fel, hogy a rekurzív fa t-edik szintje a legnagyobb, ahol a teljes részszámítás elfér a gyorsítótárban. A becsléshez így feltesszük, hogy az t-edik szinten minden csúcsban legfeljebb Z/L memóriaműveletet végzünk, az t-edik szint alatt minden csúcs nulla további memóriaművelettel jár.

Tegyük fel, hogy a $[0, a^h)$ intervallumot szitáljuk. Legyen $n = a^h$, és m a minket érdeklő prímek felső korlátja. Legyen $p \leq m$ egy prím. Legyen $C_{p,t}$ azon csúcsok száma, ahol a p prímnek van többszöröse a csúcshoz tartozó intervallumban, és a csúcs legalább a t-edik szinten van. $C_{p,t}$-t megbecsüljük C_p-hez hasonlóan, a csúcsokhoz tartozó intervallumok hossza alapján.

$$C_{p,t} \leq \sum_{k=\max(\lceil \log_a p \rceil, t)}^{h} a^{h-k} + \sum_{k=t}^{\lceil log_a p \rceil - 1} \left\lfloor \frac{n}{p} \right\rfloor$$

Vizsgáljuk az összeg baloldalát.

- Ha $\max(\lceil \log_a p \rceil, t) = \lceil \log_a p \rceil$, akkor $C_{p,t}$-re használhatjuk a C_p-re adott becslést részeredményét.

$$\sum_{k=\max(\lceil \log_a p \rceil, t)}^{h} a^{h-k} < \frac{2n}{p}$$

- Tegyük fel, hogy $\max(\lceil \log_a p \rceil, t) = t$. Ekkor

$$\begin{aligned}
\sum_{k=\max(\lceil \log_a p \rceil, t)}^{h} a^{h-k} &= \sum_{k=t}^{h} a^{h-k} \\
&= \sum_{k=0}^{h-t} a^{k} \\
&= \frac{a^{h-t+1} - 1}{a - 1} \\
&< \frac{a^{h-t+1}}{a - 1} \\
&\leq \frac{2a^{h-t+1}}{a} \\
&= \frac{2n}{a^t}
\end{aligned}$$

Vizsgáljuk az összeg jobboldalát. Tegyük fel, hogy az összeg nem üres.

$$\sum_{k=t}^{\lceil \log_a p \rceil - 1} \left\lfloor \frac{n}{p} \right\rfloor = (\lceil \log_a p \rceil - t) \left\lfloor \frac{n}{p} \right\rfloor < (\log_a p - t + 1)\frac{n}{p}$$

Legyen C_t az összes prím által bejárt csúcs száma a rekurzív fában a t-edik szinten és felette. Mivel t egy egész szám, ezért

$$\lceil \log_a p \rceil \leq t \Leftrightarrow \log_a p \leq t \Leftrightarrow p \leq a^t$$

Tegyük fel, hogy $t \geq 3$, és $m > a^t$.

$$\begin{aligned}
C_t &= \sum_{p \leq m} C_{p,t} \\
&= \sum_{p \leq a^t} C_{p,t} + \sum_{a^t < p \leq m} C_{p,t} \\
&< \sum_{p \leq a^t} \frac{2n}{a^t} + \sum_{a^t < p \leq m} \left(\frac{2n}{p} + (\log_a p - t + 1)\frac{n}{p} \right) \\
&= \frac{2n}{a^t} \sum_{p \leq a^t} 1 + \sum_{a^t < p \leq m} \left(\frac{2n}{p} + (\log_a p - t + 1)\frac{n}{p} \right) \\
&= \frac{2n}{a^t} O\left(\frac{a^t}{\ln a^t} \right) + \sum_{a^t < p \leq m} \left(\frac{2n}{p} + (\log_a p - t + 1)\frac{n}{p} \right) \\
&= O\left(\frac{n}{\ln a^t} \right) + \sum_{a^t < p \leq m} \left(\frac{2n}{p} + (\log_a p - t + 1)\frac{n}{p} \right) \\
&= O\left(\frac{n}{\ln a^t} \right) + \sum_{a^t < p \leq m} (\log_a p - t + 3)\frac{n}{p} \\
&= O\left(\frac{n}{\ln a^t} \right) + \sum_{a^t < p \leq m} \frac{n \log_a p}{p} - \sum_{a^t < p \leq m} \frac{n(t-3)}{p} \\
&= O\left(\frac{n}{\ln a^t} \right) + \frac{n}{\ln a} \sum_{a^t < p \leq m} \frac{\ln p}{p} - n(t-3) \sum_{a^t < p \leq m} \frac{1}{p} \\
&= O\left(\frac{n}{\ln a^t} \right) + \frac{n}{\ln a} \Big(\ln m - \ln a^t + O(1) \Big) - n(t-3) \sum_{a^t < p \leq m} \frac{1}{p} \\
&= O\left(\frac{n}{\ln a^t} + \frac{n}{\ln a} \right) + \frac{n}{\ln a} \ln \frac{m}{a^t} - n(t-3) \sum_{a^t < p \leq m} \frac{1}{p} \\
&= O\left(\frac{n}{\ln a} \right) + n \log_a \frac{m}{a^t} - n(t-3) \sum_{a^t < p \leq m} \frac{1}{p} \\
&\leq O\left(\frac{n}{\ln a} \right) + n \log_a \frac{m}{a^t} \\
&= O\left(n \log_a \frac{m}{a^t} \right)
\end{aligned}$$

Becsüljük meg t-t az alapján, hogy egy a^t hosszú intervallumban mennyi a tel-

jesen egyenletesen elosztott prímek súlya. Az a^t-nél nem nagyobb prímeket egy súllyal, a nagyobbakat töredék súllyal vesszük figyelembe. Legyen P_t az összes prím súlya egy a^t hosszú intervallumban. Tegyük fel, hogy $m = n^q = a^{hq}$, ahol $q \in \{\frac{1}{2}, q\}$.

$$
\begin{aligned}
P_t &= \sum_{p \le a^t} 1 + \sum_{a^t < p \le m} \left\lfloor \frac{n}{p} \right\rfloor \frac{a^t}{n} \\
&= \Theta\left(\frac{a^t}{\ln a^t}\right) + \sum_{a^t < p \le m} \left\lfloor \frac{n}{p} \right\rfloor \frac{a^t}{n} \\
&= \Theta\left(\frac{a^t}{\ln a^t}\right) + \frac{a^t}{n} \sum_{a^t < p \le m} \left\lfloor \frac{n}{p} \right\rfloor \\
&= \Theta\left(\frac{a^t}{\ln a^t}\right) + \frac{a^t}{n} \sum_{a^t < p \le m} \left(\frac{n}{p} + \Theta(1)\right) \\
&= \Theta\left(\frac{a^t}{\ln a^t}\right) + a^t \sum_{a^t < p \le m} \frac{1}{p} + \frac{a^t}{n} \sum_{a^t < p \le m} \Theta(1) \\
&= \Theta\left(\frac{a^t}{\ln a^t}\right) + a^t \left(\ln\ln m - \ln\ln a^t + \Theta(1)\right) + \frac{a^t}{n} \sum_{a^t < p \le m} \Theta(1) \\
&= \Theta\left(\frac{a^t}{\ln a^t} + a^t\right) + a^t \left(\ln\ln m - \ln\ln a^t\right) + \frac{a^t}{n} \sum_{a^t < p \le m} \Theta(1) \\
&= \Theta\left(a^t\right) + a^t \left(\ln\ln m - \ln\ln a^t\right) + \frac{a^t}{n} \sum_{a^t < p \le m} \Theta(1) \\
&= \Theta\left(a^t\right) + a^t \left(\ln\ln a^{hq} - \ln\ln a^t\right) + \frac{a^t}{n} \sum_{a^t < p \le m} \Theta(1) \\
&= \Theta\left(a^t\right) + a^t \left(\ln\left(hq \ln a\right) - \ln\left(t \ln a\right)\right) + \frac{a^t}{n} \sum_{a^t < p \le m} \Theta(1) \\
&= \Theta\left(a^t\right) + a^t \left(\ln\left(hq\right) + \ln a - \ln t - \ln a\right) + \frac{a^t}{n} \sum_{a^t < p \le m} \Theta(1) \\
&= \Theta\left(a^t\right) + a^t \left(\ln\left(hq\right) - \ln t\right) + \frac{a^t}{n} \sum_{a^t < p \le m} \Theta(1) \\
&= \Theta\left(a^t\right) + a^t \ln \frac{hq}{t} + \frac{a^t}{n} \sum_{a^t < p \le m} \Theta(1) \\
&= \Theta\left(a^t\right) + a^t \ln \frac{hq}{t} + \frac{a^t}{a^h} \Theta\left(\frac{a^{hq}}{hq \ln a}\right) \\
&= \Theta\left(a^t\right) + a^t \ln \frac{hq}{t} + \Theta\left(\frac{a^{t+h(q-1)}}{hq \ln a}\right) \\
&= \Theta\left(a^t\right) + a^t \ln \frac{hq}{t} \\
&= \Theta\left(a^t \ln \frac{q \ln n}{t}\right)
\end{aligned}
$$

Tegyük fel, hogy $Z = \Omega\left(n^d\right)$, valamilyen $0 < d \leq 1$ konstansra. Ekkor t-t megválaszthatjuk úgy, hogy $t = \log_a(\Theta(Z))$, és $Z = \Theta(P_t)$. Ezeket a C_t-re adott becslésbe helyettesítve becslést kapunk a prímek számára, amik memóriaművelettel járó csúcsot járnak be.

$$C_t = O\left(n \log_a \frac{m}{a^t}\right) = O\left(n \log_a \frac{n^q}{Z}\right)$$

Legyen N_t a csúcsok száma a t-edik szinten és fölött.

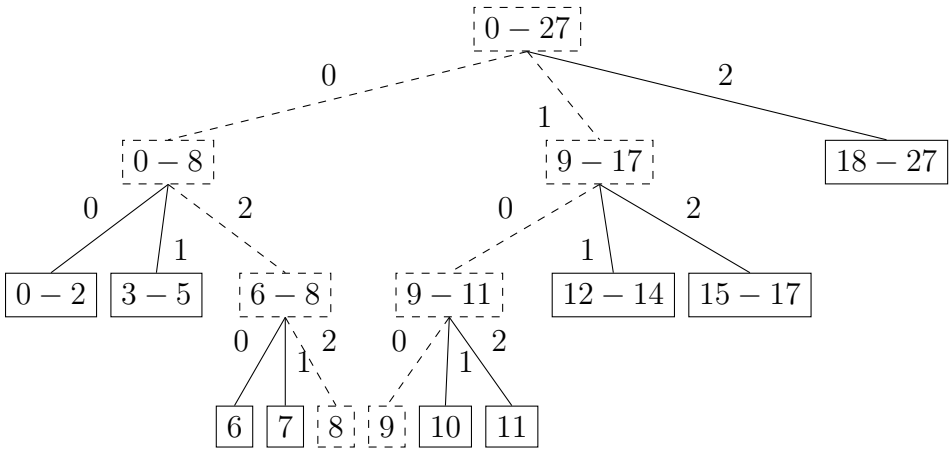
$$N_t = \sum_{k=t}^{h} a^{h-k} = \sum_{k=0}^{h-t} a^k = \frac{a^{h-t+1}-1}{a-1} < \frac{a^{h-t+1}}{a-1} \leq \frac{2a^{h-t+1}}{a} = \frac{2n}{a^t} = \Theta\left(\frac{n}{Z}\right)$$

Tehát a program a futása során $\Theta\left(\frac{n}{Z}\right)$ darab listát particionál, és az összes lista hosszának összege $O\left(n \log_a \frac{n^q}{Z}\right)$. Felhasználva, hogy egy P hosszú lista particionálása $O(1 + P/L)$ memóriaművelettel jár, az összes memóriaművelet száma gyorsítótárral

$$O\left(\frac{n}{L} \log_a \frac{n^q}{Z} + \frac{n}{Z}\right)$$

3.4.8. Edénysor szita

A RekurzívSzita által generált fát a levelek közti utakon keresztül is be lehet járni. A fában az éleket címkézzük meg azzal a számjeggyel, hogy a szülő csúcshoz tartozó részintervallum hányadik részintervalluma a gyerek csúcs. Ekkor egy út a fa gyökerétől egy levélbe a levélhez tartozó számot írja le az a alapú számrendszerben. Az a alapú számrendszerben az eggyel növelés műveletet elvégezve megkapjuk a levél in-order rákövetkezőjét.



3.5. ábra. Az út 8 és 9 között a hármas alapú számrendszerben.

Ez alapján egy prioritásos sorral elvégezhetjük a fa bejárását tetszőleges n-ig. A

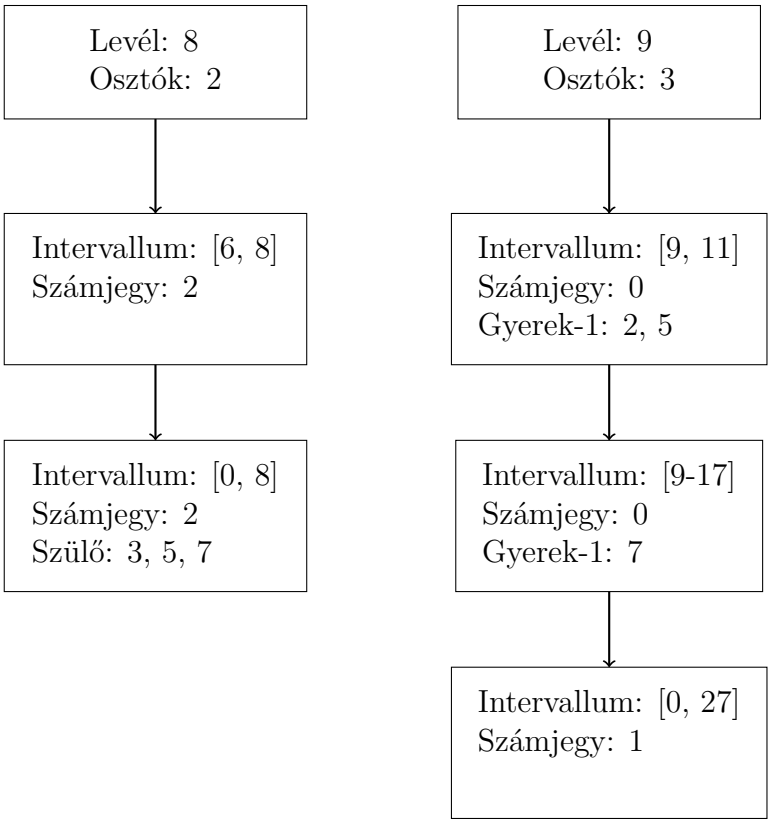
bejárást levélről levélre tesszük, in-order sorrendben. Legyen a program állapota a következő.

- Az aktuális levélhez tartozó szám.
- Egy lista az aktuális levelet osztó prímeknek.
- Egy verem, ami a levélből a gyökérbe vezető utat írja le, az a alapú számrendszerben. A verem tetején a legkisebb helyiértékű számjegy van, és a helyiértékek növekednek a veremben lefele haladva. Egy veremelem állapota legyen:
  - A veremelemhez tartozó csúcs intervallumát.
  - Egy számjegy az a alapú számrendszerben. Ez írja le az út következő élét.
  - a darab lista a prímek szétválogatásához, amikor a verem-elemhez tartozó csúcsot meglátogatjuk.
  - Egy darab lista azoknak a prímeknek, amiknek ebben a részfában már nincs többszörösük, és a fa egy felsőbb szintjén kell szétosztani.

A verem segítségével a fát egy tetszőleges leveléig bejárhatjuk. Ehhez a veremben tárolt a alapú számrendszerbeli számot egy ciklusban eggyel megnöveljük. A megnövelés két részből áll.

- Az első rész addig veszi ki az elemeket a veremből, amíg van átvitel. Ez a ciklus azt is megszámolja, hogy hány elemet vett ki a veremből összesen.
  - Kiveszi a verem tetején lévő elemet.
  - Ha nincs ilyen elem, akkor generál egy új verem-elemet, amiben a számjegy 0, és minden lista üres.
  - A veremelemben elvégzi azt a szétosztást, ami a rekurzív fában a gyerekcsúcsból visszatéréskor szükséges.
  - A veremelemben található számjegyből tudja, hogy melyik gyerekcsúcsból tér vissza.
  - A prímeket szétosztja a maradék gyerekekhez és a szülőhöz tartozó listák között.
  - Megnövel az elemben tárolt számjegyet.
  - Ha a számjegy túlcsordul, akkor az további átvitelt jelent, és a verem feldolgozását folytatja.
  - Ha a számjegy nem csordul túl, akkor a veremelemet visszateszi a verembe, és a számjegynövelés második fázisával folytatja.

- A második fázis a fában a leszálló ág. Annyi veremelemet kell a verembe vissza tenni, ahányat az első fázis kivett.
  - Jegyezzük meg a verem tetején lévő elem annyiadik listáját, amennyi a veremelemben a szám. A veremelemben töröljük a listát.
  - Készítsünk egy éj veremelemet, 0 számjeggyel, és üres listákkal.
  - A prímek megjegyzett listáját osszuk szét az új veremelem listái között, ahogy a rekurzív fában is tettük.
  - Tegyük a veremelemet a verembe.



3.6. ábra. Az edénysor állapota a 8 és a 9 levélben, hármas számrendszerben.

Az edénysor kezdőállapota minden $a \leq 2$ alapra az 1-hez tartozó levél, a prímek üres listája, és egy egyelemű verem, ahol a veremelem intervalluma $[0, a-1]$, a számjegy 1, és minden lista üres.

A RekurzívSzita program idő-, és gyorsítótár bonyolultságát csak $n = a^h$ pontokon becsültük meg. Az edénysorral az $[1, a^h - 1]$ intervallumot bejárva a RekurzívSzita által generált fát járjuk be, kivéve a gyökér $\to 0$, a $0 \to 1$, és a $a^h - 1 \to$ gyökér utat. Így a RekurzívSzitára adott bonyolultságbecslések az edénysor szitára is érvényesek az a egész hatványaiban.

Legyen $f(n)$ az edénysorszita idő-, vagy gyorsítótár bonyolultsága n függvényében. Ez rögzített a-ra, és gyorsítótár méretre $f = \Theta(n \ln n)$ a egész hatványaiban. f

monoton nő. Tegyük fel, hogy c_1 és c_2 f aszimptotikus korlátjának konstans tagjai, tehát minden egész h-ra:

$$c_1 a^h \ln a^h \leq f(a^h) \leq c_2 a^h \ln a^h$$

Legyen $f'(n) = f(a^{\lceil \log_a n \rceil})$ minden n-re, azaz f' felülbecsli f-et a következő a hatványban felvett értékkel. Legyen n olyan, hogy $a^k < n < a^{k+1}$ valamely $k > 0$ egészre. Ekkor

$$\frac{f'(n)}{f(n)} < \frac{f(a^{k+1})}{f(a^k)} \leq \frac{c_2 a^{k+1} \ln a^{k+1}}{c_1 a^k \ln a^k} = \frac{c_2 a(k+1)}{c_1 k} \leq \frac{2c_2 a}{c_1}$$

Tehát az $\frac{f'}{f}$ hányados felülről becsülhető egy konstanssal, és az edénysor szita aszimptotikus idő-, és gyorsítótár bonyolultsága megegyezik a RekurzívSzita bonyolultságával.

4. fejezet

Szimulátor

A gyorsítótár-független algoritmusok valódi viselkedésének ellenőrzéséhez szimulátort készítettem. Ennek több része van.

- A megvizsgált algoritmusok C-ben írt programok.
- Egy RISC-V emulátor, ami egy program futtatása közben elmenti a memória hozzáféréseket.
- Egy gyorsítótár szimulátor, ami egy memória-hozzáférés sorozatot a gyorsítótáron visszajátszva egy új memória-hozzáférés sorozatot készít.
- Egy összesítő, ami egy memória-hozzáférés sorozatból statisztikákat készít.

4.1. C programok

A C programokban a nyelv minden konstrukciója használható, de a sztenderd nyelvi könyvtár helyett csak néhány egyszerűsített funkció támogatott.

- A programot függőségek nélkül, statikusan, egy önálló programként kell fordítani.
  - A kimenet egy futtatható ELF (Executable and Linkable Format) formátumú bináris fájl kell legyen.
  - Az architektúra RV64IMFD[14] kell legyen.
  - A hívási konvenció LP64D[15] kell legyen.
  - A programszerkesztésnél a szegmenseket a $0x10000$ címtől kell folytonosan elrendezni.
- A belépési pont egy paraméterek nélküli függvény kell legyen. A program futása véget ér, ha ez a függvény vissza tér. Ekkor a kilépési kód nulla.

- A környezettel a következő funkciókkal lehet kommunikálni.
  - `void exit(int)`:
    befejezi a program futását, és a megadott paramétert adja vissza kilépési kódként.
  - `void *malloc(uint64_t)`:
    a memóriából lefoglal egy, a megadott méretű, darabot. Hiba esetén `NULL`-t ad vissza.
  - `void free(void*)`:
    felszabadítja a megadott, `malloc` által lefoglalt memóriát.
  - `void memory_access_log_disable()`:
    letiltja a memória-műveletek naplózását.
  - `void memory_access_log_enable()`:
    engedélyezi a memória-műveletek naplózását.
  - `void memory_access_log_user_data(uint64_t)`:
    a memória-műveletek naplójába írja a megadott értéket.
  - Műveletek a sztenderd bemenetről olvasáshoz:
    * `double read_double()`
    * `float read_float()`
    * `int16_t read_int16()`
    * `int8_t read_int8()`
    * `int32_t read_int32()`
    * `int64_t read_int64()`
    * `uint16_t read_uint16()`
    * `uint8_t read_uint8()`
    * `uint32_t read_uint32()`
    * `uint64_t read_uint64()`
  - Műveletek a sztenderd bemenetre íráshoz:
    * `write_double(double)`
    * `write_float(float)`
    * `write_int16(int16_t)`
    * `write_int8(int8_t)`
    * `write_int32(int32_t)`
    * `write_int64(int64_t)`
    * `write_uint16(uint16_t)`

    * `write_uint8(uint8_t)`
    * `write_uint32(uint32_t)`
    * `write_uint64(uint64_t)`

4.1.1. Mátrix szorzás

A mátrixszorzást megvalósításának forráskódjai:

- `matrix-multiplication-by-definition.c`
- `matrix-multiplication-halving.c`
- `matrix-multiplication-power-of-2.c`
- `matrix-multiplication-power-of-2-strassen.c`

A mátrixszorzások bemenete a következő.

- Három egész. Az első a bal mátrix sorainak száma. A második a bal mátrix oszlopainak és a jobb mátrix sorainak száma. A harmadik a jobb mátrix oszlopainak száma.
- A bal mátrix elemei sorfolytonosan felsorolva. Az értékek lebegőpontosak.
- A jobb mátrix elemei sorfolytonosan felsorolva. Az értékek lebegőpontosak.

A programok kimenetei az eredménymátrix elemei sorfolytonosan felsorolva.

Például egy szorzat kiszámításának bemenete és kimenete a következő. Az első hat sor a bement. A utolsó két sor a kimenet. A példában az új-sor karakterek szóközzel vannak helyettesítve a jobb értelmezhetőség kedvéért.

4.1. táblázat. A mátrixszorzás futtatásának példája

```
matrix-multiplication-by-definition.native-gcc-O2.elf
2 3 2
1 2 3
4 5 6
7 8
9 10
11 12
58.000000 64.000000
139.000000 154.000000
```

Egy mátrixszorzás kilépési kódja a következőek egyike lehet.

- 0: Siker.
- 1: Nincs elég szabad memória.
- 2: A mátrixok egyik dimenziója nagyobb, mint 2^{63}.

4.1.2. Rendezés

A rendezések megvalósításának forráskódjai:

- `funnelsort.c`
- `mergesort.c`

A rendezések bemenetei egész számok.

- Az első szám a rendezendő elemek száma.
- A többi szám a rendezendő elemek.

A rendezések kimenete a rendezett elemek felsorolva.

Például egy rendezés bemenete és kimenete a következő. Az első két sor a bement. A utolsó sor a kimenet. A példában az új-sor karakterek szóközzel vannak helyettesítve a jobb értelmezhetőség kedvéért.

4.2. táblázat. A rendezés futtatásának példája

```
funnelsort.native-gcc-O2.elf
5
2 5 3 1 4
1 2 3 4 5
```

Egy rendezések kilépési kódja a következőek egyike lehet.

- 0: Siker.
- 1: Nincs elég szabad memória.

4.1.3. RekurzívSzita

A RekurzívSzita megvalósításának forráskódjai:

- `bucket-sieve-cache-aware.c`
- `bucket-sieve-cache-oblivious.c`

A gyorsítótár-független RekurzívSzita bemenete két egész szám.

- Az első a rekuzív fa magassága. Ha ez a szám h, akkor a rekurzív fa a $[0, 2^h)$ intervallumot fedi le.
- A második paraméter a prímek tárolására használandó tömb mérete, bájtban.

A gyorsítótár-tudatos RekurzívSzita bemenete három egész szám.

- Az első a számrendszer alapja, azaz egy belső csúcs gyerekeinek száma a rekurzív fában. Legyen ez a paraméter a.
- Az második a rekuzív fa magassága. Ha ez a szám h, akkor a rekurzív fa a $[0, a^h)$ intervallumot fedi le.
- A harmadik paraméter a láncolt lista elemeinek lapmérete.

A RekurzívSzita kimenete a prímek listája a szitált intervallumban, növekvő sorrendben.

Például a gyorsítótár-független RekurzívSzita bemenete és kimenete a következő. A példában az új-sor karakterek szóközzel vannak helyettesítve a jobb értelmezhetőség kedvéért.

4.3. táblázat. A gyorsítótár-független RekurzívSzita futtatásának példája

```
bucket-sieve-cache-oblivious.native-gcc-O2.elf
5 10000
2 3 5 7 11 13 17 19 23 29 31
```

A következő egy példa a gyorsítótár-tudatos RekurzívSzita be- és kimenetére.

4.4. táblázat. A gyorsítótár-tudatos RekurzívSzita futtatásának példája

```
bucket-sieve-cache-aware.native-gcc-O2.elf
3 3 4096
2 3 5 7 11 13 17 19 23
```

Egy RekurzívSzita kilépési kódja a következőek egyike lehet.

- 0: Siker.
- 1: Nincs elég szabad memória.
- 2: A fa magassága nagyobb, mint 63.
- 3: A számrendszer alapja kisebb, mint 2.
- 4: A lapméret kisebb, mint 32.

4.2. RISC-V szimulátor

A szimulátor egy Java nyelven írt parancssori program. Ez tartalmazza a RISC-V szimulátort, a gyorsítótár szimulátort, és az összesített statisztikai adatok előállítását.

A szimulátor képes futtatni egy RISC-V programot egy egyszerűsített számítógépben.

- Szimulálja a RISC-V végrehajtóegység működését.
- A memóriát a gazdarendszer virtuális memóriájával szimulálja.
- Naplózza a memória-hozzáféréseket.
- Implementálja egy egyszerűsített operációs rendszer funkcióit.
  - Betölti a programot.
  - Futtatja a program, a terminálási feltétel eléréséig.
  - Kezeli a memóriafoglalásokat, és a vermet.
  - Kezeli a be-, és kimenetet.

A végrehajtó egység az RV64IMFD[14] architektúrát implementálja. Ez 64 bites RISC-V architektúra, szorzással, osztással, valamint egyszeres-, és kétszeres-pontosságú lebegőpontos számokkal. Ez elegendő egyszálas felhasználói programok futtatására.

A szimulált memória háttere lehet java heap memória, és a gazda operációs rendszer virtuális memóriája is, memória-leképzett fájlon keresztül. A memóriacímek 48 legkisebb helyiértékű bitjei használhatóak, így a memória maximális mérete 256TB lehet.

A memória-hozzáférésekről napló készül. A napló egy idősoros bináris fájl, melyben minden bejegyzés 64 bit. A következő bejegyzések lehetségesek:

- Memória-hozzáférés. Tartalmazza:
  - A hozzáférés címét.
  - A hozzáférés méretét, bájtokban.
  - A hozzáférés típusát, ami lehet utasítás betöltés, adat betöltés, és adat tárolás.
- A hozzáférési naplózás leállításának vagy elindításának kérését.
- Az eddig futtatott gépi kódú utasítások számát. Ez az érték a szimulált időnek felel meg.
- Felhasználó által generált bejegyzés, 61 bit adattartalommal.

A memóriát a szimulátor 5 összefüggő részre osztja. Ezek sorrendben:

- A szimulátor számára fenntartott rész, $0x0 - 0x10000$ intervallumban.
- Az objektumkód számára fenntartott rész. A szimulátor ide tölti be a programot. $0x10000$ címen kezdődik, és a vége a programszegmensek végeinek legnagyobbika.

- A `malloc()`-kal allokált memória.
- A szabad memória.
- A verem, ami a memória végén kezdődik, és lefelé nő.

A `malloc()` a szabad memória elejéről foglal, az allokált terület megnövelésével. A `free()` nem csinál semmit. A verem a szabad terület végéről foglal, a veremmutató csökkentésével. Ha nincs elegendő szabad memória, akkor a `malloc()` `NULL`-t ad vissza, míg a verem növelése veremtúlcsordulás hibával megállítja a szimulációt.

A szimulált operációs-rendszer hívások megszakításokkal vannak implementálva. A megszakításkezelők előre rögzített címekhez vannak kötve, és a végrehajtó egység utasítás-dekódere ezeken a címeken a memóriában tárolt utasítás helyett a megszakításkezelőre bízza a végrehajtást. A megszakításkezelők a hívási konvenció szerinti függvényként viselkednek. A paramétereket és a visszatérési értéket a regisztereken és a vermen keresztül adják át, és a művelet elvégzése után függvényként visszatérnek.

- `void exit(int)`, `void exit_ok()`: A szimulátor a program indításakor a kilépési kódot üresre állítja, és a programot addig futtatja, amíg be nem állítja az `exit` függvényekkel a kilépési kódot. Az `exit()` a paraméterként megadott számot állítja be kilépési kódnak, még a `exit_ok()` mindig nullát. A exit_ok() címét a program indításakor a belépési pont visszatérési címeként állítja be a szimulátor, így a belépési pont függvényéből visszatérve nullára állítódik a kilépési kód.
- `void memory_access_log_disable()`,
  `void memory_access_log_enable()`: Letiltja és engedélyezi a memóriahozzáférések és a felhasználói adatok naplózását a napló fájlba. Kezdetben le van tiltva.
- `memory_access_log_user_data(uint64_t)`: Naplózza a megadott felhasználói adatot a napló fájlba, ha a naplózás engedélyezve van.
- `void *malloc(uint64_t)`: Lefoglal a szabad memóriából egy legalább akkor darabot, mint a megadott paraméter. A szabad memória a szabad memória mutató és a veremmutató közötti memóriaszakasz. Ha nincs elég szabad memória, akkor `NULL`-lal tér vissza. Ha van elég szabad memória, akkor a lefoglalt memóriaszakasz elejére mutató mutatóval. A memóriát mindig a szabad memória szakasz elején foglalja le, és megnöveli szabad memória mutatót.
- `void free(void*)`: Nem csinál semmit.

- A beolvasó is kiíró műveletek a szimulátor parancssorának sztenderd ki és bemenetét használják. A kiírás minden értéket egy új sorba ír, és a formázáshoz java alapértelmezett primitív-érték formázóit használja. A beolvasás mindig egész sorokat olvas, és egy sorból egy értéket próbál értelmezni, java alapértelmezett primitív-érték elemzőivel.

A RISC-V szimulátort a mellékelt script-tel lehet elindítani:

```
run-emulator.sh \
run \
program-image \
log-file \
memory-size \
[memory-file]
```

A paraméterek a következőek.

- `program-image`: elérési út kell a futtatni kívánt ELF bináris fájlhoz.
- `log-file`: elérési út a memória-hozzáférés naplóhoz. A fájlt a szimulátor minden futtatásnál felülírja.
- `memory-size`: A szimulált memória mérete. Mértékegység nélkül bájtban értendő. Lehetséges mértékegységek a `kb`, `mb`, és `gb`, ahol a szorzók sorra 2^{10}, 2^{20}, és 2^{30}. A mértékegységet elválasztás nélkül közvetlenül egy szám után kell írni, például `"16mb"`.
- `memory-file`: Elérési út a szimulált memória hátterének memória-leképzett fájljához. A szimulátor induláskor a fájl méretét megnöveli a kívánt memória-méretre, ha az annál kisebb. Ha a paraméter meg van adva, akkor a szimulátor a java heap memória helyett a gazda operációs rendszer virtuális memóriáját használja.

4.3. Gyorsítótár szimulátor

A gyorsítótár szimulátor egy memória-elérési naplót visszajátszik egy szimulált gyorsítótáron, és a gyorsítótár által generált memória-eléréseket egy új naplóba írja. A szimulációt minden esetben egy üres gyorsítótárral kezdi. A szimuláció a napló azon bejegyzéseit, amik nem memória-elérések, az új naplóba másolja.

Egy OPT gyorsítótár szimulációját a következő paranccsal lehet elindítani.

```
run-emulator.sh \
  cache \
```

```
  opt \
  cache-size \
  line-size \
  access-type \
  input-log-file \
  output-log-file \
  temp-file
```

Az OPT gyorsítótár teljesen asszociatív. A paraméterek a következőek.

- `cache-size`: A gyorsítótár mérete gyorsítótár-sorokban.
- `line-size`: Egy gyorsítótár-sor mérete bájtokban. Az érték csak kettő hatványa lehet.
- `access-type`: A memória-elérés típusa. A következő értékek lehetnek.
  - `both`: Az utasítás betöltéseket, valamint az adat betöltéseket és tárolásokat is visszajátssza a gyorsítótárnak.
  - `data`: Az adat betöltéseket és tárolásokat visszajátssza a gyorsítótárnak, az utasítás betöltéseket átmásolja az új naplóba másolja.
  - `instruction`: Az utasítás betöltéseket visszajátssza a gyorsítótárnak, az adat betöltéseket és tárolásokat az új naplóba másolja.
- `input-log-file`: Elérési út a bemeneti naplófájlhoz.
- `output-log-file`: Elérési út a kimeneti naplófájlhoz.
- `temp-file`: Elérési út egy ideiglenes fájlhoz.

Az OPT gyorsítótár egy kétlépéses algoritmussal[5] számítja ki az eredményt. Első lépésben a naplót visszafele végigolvasva LRU-ra hasonlító módon minden memória-elérésnél meghatározza, hogy mikor lesz legközelebb ugyanez a cím elérve. Ehhez a lépéshez minden elért memóriacímet a memóriában kell tartani. A második lépésben a naplóban előrefelé olvasva szimulálja a gyorsítótárat, az első lépésben meghatározott segédadatokkal.

Egy online gyorsítótár szimulációját a következőképp lehet indítani.

```
run-emulator.sh \
  cache \
  replacement-policy \
  cache-size \
  associativity \
```

```
line-size \
access-type \
write-miss \
write-policy \
input-log-file \
output-log-file \
[random-seed]
```

A `cache-size`, `line-size`, `access-type`, `input-log-file`, és `output-log-file` paraméterek megegyeznek az OPT gyorsítótáréval. Az új paraméterek a következőek.

- `replacement-policy`: A gyorsítótár cserestratégiája. A lehetséges értékek `fifo`, `lfu`, `lru`, és `random`.
- `cache-size`: A gyorsítótár mérete gyorsítótár-sorokban. Az érték csak kettő-hatvány lehet.
- `associativity`: A gyorsítótár asszociativitása. Az érték csak kettő-hatvány lehet, és a cache-size osztója kell legyen. Ha az `associativity` értéke 1, akkor a gyorsítótár közvetlen leképezésű, és minden cserestratégiával ugyanúgy viselkedik. Ha az `associativity` és a `cache-size` egyenlő, akkor a gyorsítótár teljesen asszociatív.
- `write-miss`: A tárolás találati hibáinak kezelési módja.
  - `allocate`: A tároláshoz tartozó sor a gyorsítótárba kerül. Ha a tárolás nem fed le egy teljes memóriasort, akkor először azt betölti.
  - `dont-allocate`: A tárolást a gyorsítótárt megkerülésével közvetlenül a memóriában végzi el, a tároláshoz tartozó memóriasor nem kerül a gyorsítótárba.
- `write-policy`: Az írási stratégia.
  - `back`: A tárolást csak a gyorsítótárban végzi el, és az érintett sort megjelöli módosítottnak. Egy módosított sort csere előtt a memóriába írja.
  - `through`: A tárolást elvégzi a gyorsítótárban is, és közvetlenül a memóriában is.
- `random-seed`: A véletlen cserestratégia véletlenszám-generátor inicializáló értéke. Nem kötelező paraméter.

4.4. Összesítő

Egy memória-elérési naplóból összesített statisztikák nyerhetőek ki a következő paranccsal.

```
run-emulator.sh statistics memory-log
```

A `memory-log` egy elérési út kell legyen egy napló-fájlhoz.

A program a statisztikákat a parancssor sztenderd kimenetére írja, CSV formátumban. A program a napló bejegyzéseit sorban felolvassa, és számontartja az utolsó beolvasott bejegyzésig futtatott gépi kódú utasítások számát, és a memória-elérések számát, típusonként. A napló bejárása során naplózás letiltása, naplózás engedélyezése, és felhasználói adat bejegyzést olvasva, valamint a napló végén, az összesített adatokat egy új sorba kiírja.

A kimenet oszlopai a következőek.

- A naplóbejegyzés típusa, aminek hatására a kimeneti sort generálta.
- A futtatott gépi kódú utasítások száma.
- Az utasításbetöltések száma.
- Az adatbetöltések száma.
- A tárolások száma.
- Az adatbetöltések és a tárolások száma.
- Az összes memória-elérés száma.

Például a gyorsítótár-független RekurzívSzita memória-elérési naplójának statisztikája a következő.

4.5. táblázat. Memória-elérési napló statisztikái

type	elapsed cycles	instruction loads	data loads	stores	data accesses	total accesses
access log disabled	0	0	0	0	0	0
access log enabled	32	0	0	0	0	0
262144	129119032	129095999	21162926	13736243	34899169	163995168
524288	271417020	271373596	44621021	29013074	73634095	345007691
786432	418210485	418147504	68874967	44867925	113742892	531890396
access log disabled	565902428	565820367	93300999	60874464	154175463	719995830
end	565902439	565820367	93300999	60874464	154175463	719995830

4.5. Fordítás

A C programokat a `c` könyvtárban található `compile.sh` BASH script-tel lehet lefordítani. Ez az összes programot lefordítja GCC-vel natív futtatható programként,

valamint GCC-vel és Clang-gal RISC-V architektúrára, 0, 1, és 2-es optimalizálási szinttel. A lefordított programok a `c/out` könyvtárban találhatóak

A java program a mellékelt Maven projektfájl segítségével fordítható, a `risc-v-cache-emulator` könyvtárban kiadott `mvn package` paranccsal.

4.6. Tesztek

A C programok, a RISC-V szimulátor, és a gyorsítótár szimulátorok tesztelésére JUnit tesztek készültek. Ezek 100% lefedik a C programok, a RISC-V szimulátort, és a gyorsítótár szimulátorokat.

A JUnit teszteket a `risc-v-cache-emulator` könyvtárban kiadott `mvn test` paranccsal lehet futtatni.

A C programok tesztelése kétféleképpen történik. Kis méretű bemeneteken a kimenet és az elvárt eredmény összehasonlításával. Nagy méretű bemeneten véletlenszerűen generált bemenetekkel, és az eredményt a program egyszerűsített java implementációjának eredményével hasonlítja össze.

A RISC-V szimulátor tesztelése a C programok és egy speciális C nyelven írt funkciógyűjtemény segítségével történik. Ez a funkciógyűjtemény a C nyelv konstrukcióit exportált függvényekként teszi elérhetővé. A tesztek az összes funkció meghívják, különböző bemeneti adatokkal, és az eredményt összehasonlítják az elvárt értékkel. A teszt minden C programot hatféle változatban kipróbál, GCC-vel és Clang-gal lefordítva, valamint 0, 1, és 2 optimalizálási szinttel.

A gyorsítótár szimulációk tesztelése hasonló a C programok tesztjeihez. Kis méretű inputokon az eredményt az összehasonlítja az elvárt értékkel. Nagy méretű inputokon véletlenszerű adatokkal futtatja a szimulátort, és az eredményt egy egyszerűsített implementáció eredményével hasonlítja össze.

A naplófájl statisztikáinak előállítását manuálisan teszteltem.

4.7. Dokumentáció ???

XXXXXXXXXXXXXXX

5. fejezet

Mérési eredmények

5.1. Mátrix szorzás

A 5.1 táblázat tartalmazza a gyorsítótár cserestratégiák összehasonlítását a kettő-hatvány négyzetes mátrixok gyorsítótár-független szorzásának segítségével. A gyorsítótárak teljesen asszociatívak, visszaírási stratégiával, és írási hiba esetén foglalással. A gyorsítótár mérete 2 kilobájt 16 bájtos sorokkal, és 32 kilobájt, 64 bájtos sorokkal. Az értékek a gyorsítótár által kiváltott adat-elérések száma és az $n^3/\sqrt{Z}L$ hányadosa. Ez az érték a gyorsítótár-komplexitásra adott becslés növekedési osztályának köbös tagja. A növekedési osztály $\Theta(n + n^2/L + n^3/\sqrt{Z}L)$.

5.1. táblázat. A mátrixszorzás viselkedése különböző gyorsítótárakkal

n	Z/L	OPT	LRU	FIFO	Random	LFU
64	$2k/16$	$216,096$	$313,140$	$388,737$	$492,812$	$15015,175$
64	$32k/64$	$280,461$	$445,044$	$445,220$	$486,852$	$125636,540$
128	$2k/16$	$185,890$	$281,857$	$357,441$	$461,041$	$15018,546$
128	$32k/64$	$242,317$	$322,524$	$335,059$	$407,786$	$135273,811$
256	$2k/16$	$170,447$	$266,279$	$341,750$	$445,350$	$15011,490$
256	$32k/64$	$189,590$	$260,159$	$273,093$	$347,187$	$183494,409$

A 5.2 táblázat tartalmazza az LRU gyorsítótár komplexitásának változását az asszociativitás függvényében. A program a kettő-hatvány mátrixok szorzása, minden esetben $n = 256$. Az értékek a gyorsítótár által kiváltott adatelérések és a gyorsítótár-komplexitás köbös tagjának hányadosa.

A 5.3 táblázat tartalmazza a mátrixszorzás algoritmusok viselkedését egy LRU gyorsítótáron. Minden esetben $n = 256$. A gyorsítótár teljesen asszociatív. Az értékek a gyorsítótár által kiváltott adat-elérések száma és a gyorsítótár-komplexitás növekedési osztályának köbös tagjának hányadosa.

5.2. táblázat. Az LRU viselkedése az asszociativitás függvényében

Asszociativitás	$Z/L = 2k/16$	$Z/L = 32k/64$
Közvetlen leképezésű	$2746,013$	$6924,730$
2	$1905,836$	$1764,460$
4	$1289,445$	$1233,743$
8	$603,259$	$1236,645$
16	$353,251$	$1255,808$
32	$274,765$	$586,376$
Teljes	$266,279$	$260,159$

5.3. táblázat. A mátrixszorzás algoritmusok viselkedése LRU gyorsítótáron

Algoritmus	Becslés	$Z/L = 2k/16$	$Z/L = 32k/64$
Definíció szerint	n^3	$1,521$	$0,129$
Felezés	$n^3/\sqrt{Z}L$	$353,557$	$238,680$
Kettő-hatvány	$n^3/\sqrt{Z}L$	$266,279$	$260,159$
Strassen kettő-hatvány	$n^{\log_2 7}/\sqrt{Z}L$	$5246,720$	$4423,865$

5.2. Rendezés

XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX

5.3. RekurzívSzita

A 5.4 táblázat tartalmazza a RekurzívSzita viselkedését egy teljesen asszociatív LRU gyorsítótáron. a a számrendszer alapja. A független oszlop tartalmazza a tömbök partícióján alapuló gyorsítótár-független algoritmus eredményeit, a tudatos oszlopokban a láncolt-listák partícióján alapuló gyorsítótár-tudatos szita vannak. A láncolt listák lapmérete 4096 bájt. Az értékek a gyorsítótár által kiváltott memória-elérések és a RekurzívSzita gyorsítótár-komplexitására adott becslés domináns tagjának hányadosa. A domináns tag $racnL \log_a \frac{n^q}{Z}$. q minden esetben 1.

5.4. táblázat. A RekurzívSzita viselkedése egy LRU gyorsítótáron

n	Z/L	Független	Tudatos		
		$a = 2$	$a = 2$	$a = 4$	$a = 16$
2^{19}	$2k/16$	$41,565$	$161,061$	$181,803$	$246,441$
2^{19}	$8k/32$	$32,350$	$120,550$	$140,754$	$183,951$
2^{19}	$32k/64$	$25,572$	$65,544$	$101,527$	$147,910$
2^{19}	$128k/128$	$19,853$	$27,152$	$43,555$	$152,570$
2^{20}	$2k/16$	$41,140$	$152,295$	$171,384$	$231,929$
2^{20}	$8k/32$	$32,119$	$112,779$	$130,177$	$166,765$
2^{20}	$32k/64$	$24,879$	$59,008$	$89,151$	$126,973$
2^{20}	$128k/128$	$16,904$	$23,186$	$37,808$	$113,851$

A 5.5 táblázat tartalmazza a gyorsítótár-tudatos RekurzívSzita viselkedését a lapméret és a gyorsítótár méretének függvényében. A gyorsítótár LRU cserestratégiát használ, és teljesen asszociatív. A szitált intervallom $[0, 2^20)$. A számrendszer alapja 16. Az értékek a gyorsítótár által kiváltott memória-elérések és a RekurzívSzita gyorsítótár-komplexitására adott becslés domináns tagjának hányadosa. A domináns tag $racnL \log_a \frac{n^q}{Z}$. q minden esetben 1.

5.5. táblázat. A RekurzívSzita viselkedése a lapméret függvényében

	Z/L			
Lapméret	$2k/16$	$8k/32$	$32k/64$	$128k/128$
1024	233,926	146,900	87,728	36,338
2048	233,053	159,474	111,080	60,138
4096	231,929	166,765	126,973	113,851
8192	232,165	169,630	137,247	142,363
16384	231,837	169,852	141,595	157,628

6. fejezet

Összegzés

XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX konstans faktor vs cache méret. külső tár. hierarchia. további.

7. fejezet

Irodalomjegyzék

[1] Frigo, M., Leiserson, C. E., Prokop, H., Ramachandran, S.: Cache-oblivious algorithms, ACM Transactions on Algorithms (TALG), 8(1) (2012)., 1-22.

[2] Belady, L. A.: A study of replacement algorithms for virtual storage computers. IBM Systems Journal 5, 2 (1966), 78–101.

[3] Belady, L. A.; Nelson, R. A.; Shedler, G. S.: An anomaly in space-time characteristics of certain programs running in a paging machine. Communications of the ACM, 12(6), (1969) 349–353.

[4] Fornai, P., Iványi, A.: FIFO anomaly is unbounded. (2010) arXiv preprint arXiv:1003.1336.

[5] Mattson, R.L., Gecsei, J., Slutz, D.R., Traiger, I.L.: Evaluation Techniques for Storage Hierarchies. IBM Syst. J., 9, (1970) 78-117.

[6] Aspnes, J. (1998). Competitive analysis of distributed algorithms. In: Fiat, A., Woeginger, G.J. (eds) Online Algorithms. Lecture Notes in Computer Science, vol 1442. Springer, Berlin, Heidelberg. `https://doi.org/10.1007/BFb0029567`

[7] Daniel D. Sleator and Robert E. Tarjan.: Amortized efficiency of list update and paging rules. Commun. ACM 28, 2 (Feb. 1985), 202–208.

[8] Hennessy, John L.; Patterson, David A. (2011). Computer Architecture: A Quantitative Approach. ISBN 978-0-12-383872-8.

[9] Cormen, Thomas A.; Leiserson, Charles E.; Rivest, Ronald L. (2001). Algoritmusok. ISBN 963-16-3029-3.

[10] Strassen , V. Gaussian elimination is not optimal. Numerische Mathematik 13 (1969), 354–356.

[11] Villarino, Mark. (2005). Mertens' Proof of Mertens' Theorem. Arxiv.

[12] Selberg, Atle. An Elementary Proof of the Prime-Number Theorem. Annals of Mathematics 50, no. 2 (1949): 305–13. https://doi.org/10.2307/1969455.

[13] Cover, T. M., Thomas, J. A. (2006). Elements of Information Theory (2nd ed.). Wiley-Interscience. ISBN: 978-0-471-24195-9. p.111.

[14] The RISC-V Instruction Set Manual: Volume I `https://lists.riscv.org/g/tech-unprivileged/attachment/535/0/unpriv-isa-asciidoc.pdf`

[15] RISC-V ABIs Specification `https://lists.riscv.org/g/tech-psabi/attachment/61/0/riscv-abi.pdf`

[16] Brodal, Gerth & Fagerberg, Rolf. (2002). Cache Oblivious Distribution Sweeping. BRICS Report Series. 9. 426-438. `https://doi.org/10.1007/3-540-45465-9_37`